

Performance Analysis

[Reattempt](#)[Download Certificate](#)

Detailed analysis of your performance in the AWS Certified Solutions Architect Associate - SAA C03 exam.

Answer Distribution

0

Correct

0

Incorrect

65

Unattempted

Domain Performance

Networking	0%
Security	0%
Storage	0%
Cost Optimization	0%
Compute	0%
Databases	0%
Monitoring and Observability	0%
Automation	0%

Final Score

**0/65**

Passing score: 48

Time Taken

**130m 0s**

Total time: 130 minutes



Question Analysis

Correct	0
Incorrect	0
Unattempted	65
Reviewed	0

Question-wise Analysis

Expand All

Collapse All

Question 1

Not Attempted

Networking



A solutions architect is designing a VPC architecture with multiple subnets to host EC2 instances and RDS DB instances. The architecture includes six subnets spread across two Availability Zones: a public, private, and database subnet in each zone. Only EC2 instances in the private subnets should access the RDS databases. What solution will satisfy these requirements?

- ☐ Create a new route table that excludes routes to the public subnets' CIDR blocks. Associate the route table with the database subnets.
This option controls routing but doesn't specifically address the access requirement between EC2 instances and RDS databases.
- ☐ Create a security group that denies inbound traffic from the security group assigned to instances in the public subnets. Attach the security group to the DB instances.
Security groups operate with allow rules, not deny rules, making this option ineffective.
- ☒ Create a security group that allows inbound traffic from the security group assigned to instances in the private subnets. Attach the security group to the DB instances.
This approach ensures only EC2 instances in the private subnets can access the RDS databases by allowing inbound traffic from the private subnet security group.
- ☐ Create a new peering connection between the public subnets and the private subnets. Create a different peering connection between the private subnets and the database subnets.
Peering connections are for inter-VPC communication and not necessary or appropriate for restricting access between subnets within the same VPC.

Overall Explanation:

The correct solution is to create a security group that allows inbound traffic from the security group assigned to EC2 instances in the private subnets and attach it to the RDS DB instances. This configuration ensures that only EC2 instances running within the private subnets are authorized to access the RDS databases. Security groups provide stateful filtering for precise access control at the instance level. Route tables and peering connections aren't suitable for controlling traffic between subnets in the same VPC. Security groups do not support deny rules.

[View Discussions](#)[Ask AI](#)

Question 2

Not Attempted

Security



A company is using AWS Key Management Service (AWS KMS) keys to encrypt AWS Lambda environment variables. A solutions architect needs to ensure that the required permissions are

in place to decrypt and use the environment variables. Which steps must the solutions architect take to implement the correct permissions? (Choose two.)

- ☐ Add AWS KMS permissions in the Lambda resource policy.
Adding permissions to the Lambda resource policy is not sufficient for decrypting environment variables.
- ☒ Add AWS KMS permissions in the Lambda execution role.
The Lambda execution role needs kms:Decrypt and kms:GenerateDataKey permissions to call KMS APIs.
- ☐ Add AWS KMS permissions in the Lambda function policy.
Adding permissions to the Lambda function policy is not sufficient for decrypting environment variables.
- ☒ Allow the Lambda execution role in the AWS KMS key policy.
The KMS key policy needs to allow the Lambda execution role to have kms:Decrypt and kms:GenerateDataKey permissions for that specific key.
- ☐ Allow the Lambda resource policy in the AWS KMS key policy.
Allowing the Lambda resource policy is incorrect; the execution role requires the KMS key policy permissions.

Overall Explanation:

To decrypt environment variables encrypted with AWS KMS, Lambda needs to be granted permissions to call KMS APIs. This is done by adding AWS KMS permissions to the Lambda execution role, which grants the Lambda function permissions to access other AWS services. Also, the KMS key policy needs to allow the Lambda execution role to have kms:Decrypt and kms:GenerateDataKey permissions for that specific key.

[View Discussions](#)[Ask AI](#)

Question 3

Not Attempted

[Storage](#)

A company's financial application generates reports averaging 50 KB in size and stores them in Amazon S3. These reports are accessed frequently during the first week after creation and must

be retained for several years. The reports need to be retrievable within 6 hours. Which solution meets these requirements MOST cost-effectively?

- ☒ Use S3 Standard. Use an S3 Lifecycle rule to transition the reports to S3 Glacier after 7 days.

S3 Standard provides frequent access, and S3 Glacier offers cost-effective long-term storage with retrieval times within 6 hours using standard retrieval.

- ☐ Use S3 Standard. Use an S3 Lifecycle rule to transition the reports to S3 Standard-Infrequent Access (S3 Standard-IA) after 7 days.

S3 Standard-IA is more expensive than Glacier for long-term archival and offers similar retrieval performance for frequently accessed data after the initial week. Also, S3 Standard-IA has a minimum storage duration, making Glacier more appropriate for archival after just 7 days. Furthermore, transitioning to S3 Standard-IA requires a minimum of 30 days of storage.

- ☐ Use S3 Intelligent-Tiering. Configure S3 Intelligent-Tiering to transition the reports to S3 Standard-Infrequent Access (S3 Standard-IA) and S3 Glacier.

S3 Intelligent-Tiering is not the most cost-effective solution when access patterns are known. It is designed for unknown or changing access patterns. Also, transitioning from S3 Intelligent-Tiering to S3 Standard-IA is not supported.

- ☐ Use S3 Standard. Use an S3 Lifecycle rule to transition the reports to S3 Glacier Deep Archive after 7 days.

S3 Glacier Deep Archive has retrieval times that exceed 6 hours, violating the requirement.

Overall Explanation:

S3 Standard provides immediate access for the first week, then using a Lifecycle rule to transition to S3 Glacier offers cost-effective long-term storage while meeting the 6-hour retrieval requirement. S3 Intelligent-Tiering is for unknown access patterns and is more expensive in this scenario. S3 Standard-IA and S3 Glacier Deep Archive do not fulfill the retrieval time requirements or are more expensive than S3 Glacier.

 View Discussions

 Ask AI

Question 4

Not Attempted

Cost Optimization



A company requires a solution to optimize the cost of its Amazon EC2 instances. The company anticipates the need to change the type and family of its EC2 instances every 2-3 months. Which approach best meets these requirements?

- ☐ Purchase Partial Upfront Reserved Instances for a 3-year term.
Reserved Instances lock you into an instance type and family for a term. A 3-year term is not flexible enough to meet the changing needs of the company.
- ☒ Purchase a No Upfront Compute Savings Plan for a 1-year term.
A Compute Savings Plan allows switching instance types and families freely within the term as needed. No Upfront allows cost savings without an upfront payment. A 1-year term offers a balance between commitment and flexibility.
- ☐ Purchase All Upfront Reserved Instances for a 1-year term.
Reserved Instances lock you into an instance type and family. While the term is shorter, it does not allow flexibility to change the family and type of EC2 instances every 2-3 months.
- ☐ Purchase an All Upfront EC2 Instance Savings Plan for a 1-year term.
EC2 Instance Savings Plans only provide the flexibility to change usage between instances WITHIN a family. The question states that they need to change both type and family.

Overall Explanation:

The Compute Savings Plan provides the flexibility to change instance types and families freely within the term as needed, therefore it is the best option. The 'No Upfront' option provides cost savings without an upfront commitment. Since EC2 Instance Savings Plans don't allow switching instance families, they can't be the solution.

 View Discussions

 Ask AI

Question 5

Not Attempted

Security



A solutions architect needs to review a company's Amazon S3 buckets to discover personally identifiable information (PII). The company stores the PII data in the us-east-1 Region and us-west-2 Region. Which solution will meet these requirements with the LEAST operational overhead?

- ✓ Configure Amazon Macie in each Region. Create a job to analyze the data that is in Amazon S3.

Amazon Macie is specifically designed for discovering and classifying sensitive data like PII in S3. It can be enabled directly in the required Regions and set up to automatically scan the S3 buckets on a schedule, minimizing operational overhead.

- ✗ Configure AWS Security Hub for all Regions. Create an AWS Config rule to analyze the data that is in Amazon S3.

AWS Security Hub is primarily for security monitoring across AWS accounts, not specifically for PII discovery. Configuring it for this purpose would introduce more operational overhead than necessary.

- ✗ Configure Amazon Inspector to analyze the data that is in Amazon S3.

Amazon Inspector is designed for infrastructure vulnerability management and not for PII discovery in S3 buckets. This option doesn't directly address the requirement and isn't the most efficient.

- ✗ Configure Amazon GuardDuty to analyze the data that is in Amazon S3.

Amazon GuardDuty focuses on network protection and threat detection, not PII discovery in S3 buckets. Using GuardDuty would be ineffective and introduce unnecessary overhead.

Overall Explanation:

Amazon Macie is the optimal solution for discovering and classifying sensitive data like PII in S3 due to its design and capabilities. Enabling Macie in the specified regions and creating jobs to analyze the S3 data directly addresses the requirement with the least operational overhead. Security Hub, Inspector, and GuardDuty provide broader security capabilities but are not specifically tailored for PII discovery and would require more complex configurations.

 View Discussions

 Ask AI

Question 6

Not Attempted

Compute



A company's SAP application has a backend SQL Server database in an on-premises environment. The company wants to migrate its on-premises application and database server to AWS. The company needs an instance type that meets the high demands of its SAP database. On-premises performance data shows that both the SAP application and the database have high memory utilization. Which solution will meet these requirements?

- ☐ Use the compute optimized instance family for the application. Use the memory optimized instance family for the database.
Compute optimized instances may not provide enough memory for the SAP application's needs.
- ☐ Use the storage optimized instance family for both the application and the database.
Storage optimized instances are not optimized for memory or compute, which are high demands for SAP applications.
- ☒ Use the memory optimized instance family for both the application and the database.
Since both the application and database have high memory needs, the memory optimized family like R5 instances meet those requirements well. Using the same instance family simplifies management and operations.
- ☐ Use the high performance computing (HPC) optimized instance family for the application. Use the memory optimized instance family for the database.
HPC is overprovisioned for the SAP application, and memory optimization is also critical for the application tier.

Overall Explanation:

The memory optimized instance family is the best choice because both the SAP application and the SQL Server database have high memory utilization. Using the same instance family simplifies management and operations. Compute optimized instances may not provide enough memory for the SAP application's needs. Storage optimized is overkill for the database's compute and memory needs. HPC is overprovisioned for the SAP app.

[View Discussions](#)[Ask AI](#)

Question 7

Not Attempted

Networking



A company runs an application in a VPC with public and private subnets across multiple Availability Zones. The application runs on Amazon EC2 instances in private subnets and uses an Amazon Simple Queue Service (Amazon SQS) queue. A solutions architect needs to design a secure solution to establish a connection between the EC2 instances and the SQS queue. Which solution will meet these requirements?

- ☒ Implement an interface VPC endpoint for Amazon SQS. Configure the endpoint to use the private subnets. Add to the endpoint a security group that has an inbound access rule that allows traffic from the EC2 instances that are in the private subnets.

This is the correct answer. An interface VPC endpoint provides a private connection to SQS. Configuring it to use private subnets keeps traffic within the VPC. The security group restricts traffic to only authorized EC2 instances.

- ☐ Implement an interface VPC endpoint for Amazon SQS. Configure the endpoint to use the public subnets. Attach to the interface endpoint a VPC endpoint policy that allows access from the EC2 instances that are in the private subnets.

Configuring the endpoint to use public subnets exposes the traffic to the public internet, which is not a secure solution.

- ☐ Implement an interface VPC endpoint for Amazon SQS. Configure the endpoint to use the public subnets. Attach an Amazon SQS access policy to the interface VPC endpoint that allows requests from only a specified VPC endpoint.

Using public subnets negates the benefit of a private connection. Security should be at the security group level.

- ☐ Implement a gateway endpoint for Amazon SQS. Add a NAT gateway to the private subnets. Attach an IAM role to the EC2 instances that allows access to the SQS queue.

A gateway endpoint does not support SQS. NAT gateway allows the instance to access resources on the public internet, and an IAM role allows the instances to interact with the SQS queue. However, it does not establish a private connection.

Overall Explanation:

The most secure solution involves using an interface VPC endpoint for SQS configured for private subnets, coupled with a security group that restricts access to the EC2 instances in the private subnets. This keeps all traffic within the VPC and limits access to authorized sources. Other options either expose the traffic to the public internet or use unsupported endpoint types for SQS.

 View Discussions

 Ask AI

Question 8

Not Attempted

Security



A solutions architect is using an AWS CloudFormation template to deploy a three-tier web application. The web application consists of a web tier and an application tier that stores and retrieves user data in Amazon DynamoDB tables. The web and application tiers are hosted on Amazon EC2 instances, and the database tier is not publicly accessible. The application EC2 instances need to access the DynamoDB tables without exposing API credentials in the template. What should the solutions architect do to meet these requirements?

- ⊗ Create an IAM role to read the DynamoDB tables. Associate the role with the application instances by referencing an instance profile.

This option only grants read access, while the application needs to both read and write data to DynamoDB.

- ✓ Create an IAM role that has the required permissions to read and write from the DynamoDB tables. Add the role to the EC2 instance profile, and associate the instance profile with the application instances.

This is the correct approach. By creating an IAM role with the necessary permissions and associating it with the EC2 instance profile, the instances can securely access DynamoDB without exposing credentials in the template.

- ⊗ Use the parameter section in the AWS CloudFormation template to have the user input access and secret keys from an already-created IAM user that has the required permissions to read and write from the DynamoDB tables.

This option exposes credentials to the user who creates the stack and is not a secure practice.

- ⊗ Create an IAM user in the AWS CloudFormation template that has the required permissions to read and write from the DynamoDB tables. Use the GetAtt function to retrieve the access and secret keys, and pass them to the application instances through the user data.

This option embeds credentials in the user data, which is highly insecure and not recommended.

Overall Explanation:

The best practice for allowing EC2 instances to access AWS services like DynamoDB is to use IAM roles. By creating an IAM role with the necessary permissions to read and write from DynamoDB and associating it with the EC2 instance profile, the instances can securely access DynamoDB without needing to manage or expose credentials directly. Option B correctly implements this approach. The application tier stores and retrieves data, it means read and write permissions are required. Options A doesn't have write permissions.

[View Discussions](#)[Ask AI](#)

Question 9

Not Attempted

Databases



A solutions architect manages an analytics application. The application stores large amounts of semistructured data in an Amazon S3 bucket. The solutions architect wants to use parallel data processing to process the data more quickly. The solutions architect also wants to use information that is stored in an Amazon Redshift database to enrich the data. Which solution will meet these requirements?

- ☐ Use Amazon Athena to process the S3 data. Use AWS Glue with the Amazon Redshift data to enrich the S3 data.

Athena is primarily a query service and doesn't inherently provide data processing capabilities for enriching data from other sources in a parallel manner. Also, while Glue can work with Redshift, this combination doesn't provide the most efficient parallel processing for large datasets.

- ☒ Use Amazon EMR to process the S3 data. Use Amazon EMR with the Amazon Redshift data to enrich the S3 data.

Amazon EMR is well-suited for processing large datasets in S3 using frameworks like Spark or Hadoop, enabling parallel data processing. EMR can integrate with Redshift to access and join data, facilitating data enrichment.

- ☐ Use Amazon EMR to process the S3 data. Use Amazon Kinesis Data Streams to move the S3 data into Amazon Redshift so that the data can be enriched.

Using Kinesis Data Streams to move data into Redshift adds unnecessary complexity and latency, especially since EMR can directly access and process data in both S3 and Redshift. Kinesis is more suited for real-time streaming, which is not the primary requirement here.

- ☐ Use AWS Glue to process the S3 data. Use AWS Lake Formation with the Amazon Redshift data to enrich the S3 data.

While AWS Glue can process data in S3, it might not be as efficient as EMR for very large datasets and complex processing. AWS Lake Formation primarily focuses on data governance and security and doesn't directly enrich the data itself.

Overall Explanation:

Amazon EMR is the best choice because it's designed for parallel data processing of large datasets and can seamlessly integrate with both Amazon S3 and Amazon Redshift. This allows for efficient processing of the S3 data and enrichment using data from Redshift.

Athena is more suitable for querying, and Kinesis Data Streams introduces unnecessary complexity. While Glue could be an option, EMR is often more appropriate for heavy parallel processing.

[View Discussions](#)[Ask AI](#)

Question 10

Not Attempted

Networking



A company operates two VPCs within the same AWS account and the us-east-1 Region. They require a solution to facilitate network traffic between these VPCs, with an estimated monthly data transfer of 400 GB. Which of the following options represents the MOST cost-effective method for connecting these VPCs?

- ☐ Implement AWS Transit Gateway to connect the VPCs. Update the route tables of each VPC to route traffic to the transit gateway.

AWS Transit Gateway introduces hourly charges and data processing fees, making it a less cost-effective solution for simple inter-VPC communication.

- ☐ Establish an AWS Site-to-Site VPN connection between the VPCs. Modify the route tables of each VPC to direct traffic through the VPN.

AWS Site-to-Site VPN incurs hourly connection charges and data transfer fees, rendering it more expensive than VPC peering for this use case.

- ☒ Configure a VPC peering connection between the VPCs. Update the route tables of each VPC to utilize the VPC peering connection.

VPC peering offers the most cost-effective approach for connecting VPCs within the same AWS region and account. It does not impose any additional charges beyond standard data transfer rates within the same region, which are free.

- ☐ Establish a 1 GB AWS Direct Connect connection between the VPCs. Update the route tables of each VPC to use the Direct Connect connection.

AWS Direct Connect entails substantial hourly charges and is an overkill solution for the moderate data transfer requirements outlined in the scenario. It is designed for high-bandwidth, low-latency connectivity and is not suitable for this cost-sensitive scenario.

Overall Explanation:

VPC peering stands as the most economical solution for enabling private network connectivity between two VPCs situated within the same AWS region and account. Given the monthly data transfer volume of 400 GB, the traffic falls under the free data transfer tier offered by AWS within the same region. Transit Gateway and VPN connections introduce additional hourly and data processing expenses, making them less attractive options. Direct Connect provides dedicated connectivity but is overly complex and expensive for the moderate bandwidth needs of the company. Therefore, VPC peering offers the simplest and most cost-effective means of establishing private communication between the VPCs.

[View Discussions](#)[Ask AI](#)

Question 11

Not Attempted

Cost Optimization



A company hosts multiple applications on AWS for different product lines. The applications use different compute resources, including Amazon EC2 instances and Application Load Balancers. The applications run in different AWS accounts under the same organization in AWS Organizations across multiple AWS Regions. Teams for each product line have tagged each compute resource in the individual accounts. The company wants more details about the cost for each product line from the consolidated billing feature in Organizations. Which combination of steps will meet these requirements?

- ☐ Select a specific AWS generated tag in the AWS Billing console.
AWS generated tags are predefined by AWS and won't align to product lines.
- ☒ Select a specific user-defined tag in the AWS Billing console.
User-defined tags were created by each product team to identify resources. Selecting the relevant tag in the Billing console will group costs.
- ☐ Select a specific user-defined tag in the AWS Resource Groups console.
Resource Groups helps manage resources but not billing.
- ☐ Activate the selected tag from each AWS account.
Activating the tag from each account is not needed since Organizations centralizes billing.
- ☒ Activate the selected tag from the Organizations management account.

The tag must be activated from the Organizations management account to consolidate billing across all accounts.

Overall Explanation:

To accurately track costs per product line using consolidated billing in AWS Organizations, you must first select the user-defined tags (created by the product teams) in the AWS Billing console. Then, activate these selected tags from the Organizations management account. This ensures that cost allocation data is collected across all member accounts based on the defined tags.

[View Discussions](#)[Ask AI](#)

Question 12

Not Attempted

Networking



A company has registered its domain name with Amazon Route 53. The company uses Amazon API Gateway in the ca-central-1 Region as a public interface for its backend microservice APIs. Third-party services consume the APIs securely. The company wants to design its API Gateway URL with the company's domain name and corresponding certificate so that the third-party services can use HTTPS. Which solution will meet these requirements?

- ☐ Create stage variables in API Gateway with Name="Endpoint-URL" and Value="Company Domain Name" to overwrite the default URL. Import the public certificate associated with the company's domain name into AWS Certificate Manager (ACM).

This approach is insufficient as it does not involve associating a custom domain and certificate directly with the API Gateway endpoint.

- ☐ Create Route 53 DNS records with the company's domain name. Point the alias record to the Regional API Gateway stage endpoint. Import the public certificate associated with the company's domain name into AWS Certificate Manager (ACM) in the us-east-1 Region.

Importing the certificate into ACM in the us-east-1 Region is incorrect for a Regional API Gateway in ca-central-1. Certificates must be in the same region as the API Gateway.

- ☒ Create a Regional API Gateway endpoint. Associate the API Gateway endpoint with the company's domain name. Import the public certificate associated with the company's domain name into AWS Certificate Manager (ACM) in the same Region. Attach the

certificate to the API Gateway endpoint. Configure Route 53 to route traffic to the API Gateway endpoint.

This is the correct solution. It creates a Regional API Gateway endpoint, associates it with the company's domain, imports the certificate into ACM in the same region (ca-central-1), attaches the certificate to the API Gateway endpoint, and configures Route 53 to route traffic to the API Gateway endpoint.

- ⊗ Create a Regional API Gateway endpoint. Associate the API Gateway endpoint with the company's domain name. Import the public certificate associated with the company's domain name into AWS Certificate Manager (ACM) in the us-east-1 Region. Attach the certificate to the API Gateway APIs. Create Route 53 DNS records with the company's domain name. Point an A record to the company's domain name.

Importing the certificate into ACM in the us-east-1 Region is incorrect for a Regional API Gateway in ca-central-1. Also, attaching the certificate to API Gateway APIs is not the correct approach.

Overall Explanation:

The correct solution involves creating a Regional API Gateway endpoint, associating it with the company's domain name, importing the SSL/TLS certificate into ACM in the same Region (ca-central-1), attaching the certificate to the API Gateway endpoint, and configuring Route 53 to route traffic to the endpoint. ACM certificates are regional and must reside in the same region as the API Gateway. Option C encompasses all these necessary steps.

[View Discussions](#)[Ask AI](#)

Question 13

Not Attempted

Monitoring and Observability



A company's solutions architect is designing an AWS multi-account solution that uses AWS Organizations. The solutions architect has organized the company's accounts into organizational units (OUs). The solutions architect needs a solution that will identify any changes to the OU hierarchy and notify the company's operations team of these changes. Which solution will meet these requirements with the LEAST operational overhead?

- ✔ Provision the AWS accounts by using AWS Control Tower. Use account drift notifications to identify the changes to the OU hierarchy.

AWS Control Tower simplifies multi-account setup and provides built-in account drift notifications that automatically detect changes to the OU hierarchy. This approach offers lower operational

overhead compared to other solutions.

- ⊗ Provision the AWS accounts by using AWS Control Tower. Use AWS Config aggregated rules to identify the changes to the OU hierarchy.

While AWS Config can track resource configurations, using it for OU hierarchy changes requires creating and managing aggregated rules, which increases operational overhead compared to Control Tower's built-in drift detection.

- ⊗ Use AWS Service Catalog to create accounts in Organizations. Use an AWS CloudTrail organization trail to identify the changes to the OU hierarchy.

Using CloudTrail requires parsing and analyzing log data to identify OU hierarchy changes, which adds complexity and operational overhead. AWS Service Catalog focuses on account provisioning rather than ongoing change detection.

- ⊗ Use AWS CloudFormation templates to create accounts in Organizations. Use the drift detection operation on a stack to identify the changes to the OU hierarchy.

CloudFormation drift detection focuses on changes within a CloudFormation stack, not across the entire OU hierarchy. This approach is not suitable for the given requirements and increases operational overhead.

Overall Explanation:

The best solution is to provision the AWS accounts using AWS Control Tower and use account drift notifications. Control Tower is a fully managed service that simplifies multi-account setup and provides built-in drift detection for OU changes. This approach offers the least operational overhead because it's automated and doesn't require custom rules or log analysis. Control Tower offers passive and active methods of drift monitoring protection for preventative controls.

[View Discussions](#)[Ask AI](#)

Question 14

Not Attempted

[Databases](#)

A company's website handles millions of requests each day, and the number of requests continues to increase. A solutions architect needs to improve the response time of the web application. The solutions architect determines that the application needs to decrease latency when retrieving product details from the Amazon DynamoDB table. Which solution will meet these requirements with the LEAST amount of operational overhead?

- ☒ Set up a DynamoDB Accelerator (DAX) cluster. Route all read requests through DAX.
DAX provides a DynamoDB-compatible caching layer to reduce read latency. It requires minimal application changes and handles caching logic automatically, resulting in the least operational overhead.
- ☐ Set up Amazon ElastiCache for Redis between the DynamoDB table and the web application. Route all read requests through Redis.
Using ElastiCache for Redis requires more integration work to sync DynamoDB data and more code changes, resulting in greater operational overhead than DAX.
- ☐ Set up Amazon ElastiCache for Memcached between the DynamoDB table and the web application. Route all read requests through Memcached.
Using ElastiCache for Memcached requires more integration work to sync DynamoDB data and more code changes, resulting in greater operational overhead than DAX.
- ☐ Set up Amazon DynamoDB Streams on the table, and have AWS Lambda read from the table and populate Amazon ElastiCache. Route all read requests through ElastiCache.
This solution is a complex event-driven approach requiring ongoing maintenance and complex integration, which leads to a high operational overhead.

Overall Explanation:

DynamoDB Accelerator (DAX) is purpose-built for accelerating DynamoDB workloads. It requires minimal application changes as read requests are routed through it. DAX handles caching logic automatically without needing complex integration code. Therefore, using DAX results in the least operational overhead. ElastiCache Redis/Memcached require more integration, and using Lambda and Streams is complex. Thus, DAX is the optimal choice.

[View Discussions](#)[Ask AI](#)

Question 15

Not Attempted

Networking



A solutions architect needs to ensure that API calls to Amazon DynamoDB from Amazon EC2 instances in a VPC do not traverse the internet. Which combination of steps should the solutions architect take to meet this requirement?

- ☒ Create a route table entry for the endpoint.
A route table entry directs traffic destined for DynamoDB to the gateway endpoint.

☒ Create a gateway endpoint for DynamoDB.

A gateway endpoint for DynamoDB enables private connectivity between DynamoDB and the VPC, allowing EC2 instances to access DynamoDB APIs without traversing the internet.

☐ Create an interface endpoint for Amazon EC2.

Interface endpoints are used for services like S3 and Systems Manager, not DynamoDB or EC2 itself.

☐ Create an elastic network interface for the endpoint in each of the subnets of the VPC.

Gateway endpoints do not require elastic network interfaces.

☐ Create a security group entry in the endpoint's security group to provide access.

This is not applicable as Gateway Endpoint does not have security groups.

Overall Explanation:

To ensure that API calls to DynamoDB from EC2 instances in a VPC do not traverse the internet, you need to create a gateway endpoint for DynamoDB and then create a route table entry to direct traffic to that endpoint. Gateway endpoints provide private connectivity to DynamoDB, and the route table entry ensures that traffic within the VPC destined for DynamoDB uses the endpoint.

 View Discussions

 Ask AI

Question 16

Not Attempted

Monitoring and Observability



A company runs its applications on both Amazon Elastic Kubernetes Service (Amazon EKS) clusters and on-premises Kubernetes clusters. The company wants to view all clusters and workloads from a central location with the LEAST operational overhead. Which solution meets these requirements?

☐ Use Amazon CloudWatch Container Insights to collect and group the cluster information.

Amazon CloudWatch Container Insights only provides metrics and logs, not cluster management capabilities. It doesn't provide the unified view required.

☒ Use Amazon EKS Connector to register and connect all Kubernetes clusters.

Amazon EKS Connector allows registering external Kubernetes clusters (on-premises and otherwise) with Amazon EKS. This provides a unified view and management of all clusters within the EKS console, minimizing operational overhead.

⊗ Use AWS Systems Manager to collect and view the cluster information.

AWS Systems Manager is more general purpose and does not natively integrate with EKS to provide a comprehensive view of clusters and workloads with minimal setup.

⊗ Use Amazon EKS Anywhere as the primary cluster to view the other clusters with native Kubernetes commands.

Amazon EKS Anywhere is for creating and managing EKS clusters on-premises, not for providing a single pane of glass for external clusters or centralizing the view of existing EKS and on-premises clusters.

Overall Explanation:

Amazon EKS Connector is the best solution because it allows you to register and connect any conformant Kubernetes cluster to AWS and visualize it in the Amazon EKS console. This provides a centralized view and management of all clusters while minimizing operational overhead. CloudWatch provides metrics, Systems Manager is too general, and EKS Anywhere is for on-premises EKS cluster creation, not central management of existing clusters.

 View Discussions

 Ask AI

Question 17

Not Attempted

Security



A company is building an e-commerce application and needs to store sensitive customer information, including Personally Identifiable Information (PII). The company requires customers to complete purchase transactions directly on the website. To meet compliance requirements, the company must ensure that sensitive customer data is protected, even from database administrators. Which solution meets these requirements?

⊗ Store sensitive data in an Amazon Elastic Block Store (Amazon EBS) volume. Use EBS encryption to encrypt the data. Use an IAM instance role to restrict access.

EBS encryption protects data at rest, but it doesn't prevent access from administrators. IAM roles restrict access to the instance, not necessarily to the data within the EBS volume itself.

- ✓ Store sensitive data in Amazon RDS for MySQL. Use AWS Key Management Service (AWS KMS) client-side encryption to encrypt the data.

Using client-side encryption with KMS allows encrypting sensitive data before it is stored in the database. The data remains encrypted at rest and in transit to the database. This protects the data from database administrators who do not have access to the encryption key.

- ✗ Store sensitive data in Amazon S3. Use AWS Key Management Service (AWS KMS) server-side encryption to encrypt the data. Use S3 bucket policies to restrict access.

S3 server-side encryption protects data at rest in S3, but it doesn't necessarily prevent access from administrators who have sufficient privileges. Bucket policies control access to the bucket, but not necessarily to the encrypted data itself if an administrator has the appropriate credentials.

- ✗ Store sensitive data in Amazon FSx for Windows Server. Mount the file share on application servers. Use Windows file permissions to restrict access.

Windows file permissions restrict access to files on the file share, but don't encrypt the data itself. Administrators with appropriate permissions can still access the data. This doesn't offer sufficient protection against database administrators in general.

Overall Explanation:

The best solution is to store the sensitive data in Amazon RDS for MySQL and use AWS KMS client-side encryption. Client-side encryption ensures that the data is encrypted before being stored in the database, preventing database administrators from viewing the unencrypted data, even if they have access to the database. This addresses the core requirement of protecting the data from database administrators.

 View Discussions

 Ask AI

Question 18

Not Attempted

Databases



A company has an on-premises MySQL database that handles transactional data. The company is migrating the database to the AWS Cloud. The migrated database must maintain compatibility with the company's applications that use the database. The migrated database also must scale automatically during periods of increased demand. Which migration solution will meet these requirements?

- ✗ Use native MySQL tools to migrate the database to Amazon RDS for MySQL. Configure elastic storage scaling.

RDS for MySQL does not scale as well as Aurora. Also, native MySQL tools aren't the recommended approach.

- ✗ Migrate the database to Amazon Redshift by using the mysqldump utility. Turn on Auto Scaling for the Amazon Redshift cluster.

Redshift is designed for analytics, not transactional data, and is not compatible with MySQL.

- ✓ Use AWS Database Migration Service (AWS DMS) to migrate the database to Amazon Aurora. Turn on Aurora Auto Scaling.

AWS DMS provides a smooth migration path to Aurora, minimizing downtime. Aurora maintains MySQL compatibility and offers Auto Scaling to handle increased demand.

- ✗ Use AWS Database Migration Service (AWS DMS) to migrate the database to Amazon DynamoDB. Configure an Auto Scaling policy.

DynamoDB is a NoSQL database and is not compatible with MySQL. This migration would require significant application code changes.

Overall Explanation:

The most suitable solution is to use AWS DMS to migrate the database to Amazon Aurora and then enable Aurora Auto Scaling. DMS facilitates migration, Aurora ensures compatibility with existing MySQL applications, and Aurora Auto Scaling handles increased demand automatically. Redshift is not for transactional data, RDS MySQL doesn't scale as well as Aurora, and DynamoDB is not MySQL-compatible.

[View Discussions](#)[Ask AI](#)

Question 19

Not Attempted

Storage



A company runs multiple Amazon EC2 Linux instances in a VPC across two Availability Zones. The instances host applications that use a hierarchical directory structure. The applications need to read and write rapidly and concurrently to shared storage. What should a solutions architect do to meet these requirements?

- ✗ Create an Amazon S3 bucket. Allow access from all the EC2 instances in the VPC.

Amazon S3 does not provide a hierarchical directory structure as it is an object storage service.

- ☒ Create an Amazon Elastic File System (Amazon EFS) file system. Mount the EFS file system from each EC2 instance.

Amazon EFS provides a scalable, high-performance NFS file system that can be concurrently accessed from multiple EC2 instances across multiple Availability Zones. It supports hierarchical directory structures.

- ☐ Create a file system on a Provisioned IOPS SSD (io2) Amazon Elastic Block Store (Amazon EBS) volume. Attach the EBS volume to all the EC2 instances.

EBS multi-attach is limited to a single Availability Zone, and doesn't fulfill requirement of storage across two Availability Zones.

- ☐ Create file systems on Amazon Elastic Block Store (Amazon EBS) volumes that are attached to each EC2 instance. Synchronize the EBS volumes across the different EC2 instances.

Synchronizing EBS volumes across multiple instances is complex, inefficient, and doesn't provide true shared storage with concurrent access.

Overall Explanation:

Amazon EFS is the best solution because it provides a shared, scalable file system that can be accessed concurrently by multiple EC2 instances across Availability Zones. It also supports the hierarchical directory structure required by the applications. S3 is not a file system. EBS volumes cannot be shared across AZs, and replication is inefficient and complex.

 View Discussions

 Ask AI

Question 20

Not Attempted

Databases



A solutions architect is designing a workload to store hourly energy consumption data from business tenants in a building. Sensors will send HTTP requests to a database, accumulating usage data for each tenant. The architect aims to use managed services wherever possible and anticipates adding independent components in the future. Which solution meets these requirements with the LEAST operational overhead?

- ☒ Use Amazon API Gateway with AWS Lambda functions to receive data, process it, and store it in an Amazon DynamoDB table.

This option leverages fully managed services, minimizing operational overhead. API Gateway handles HTTP requests, Lambda provides serverless compute, and DynamoDB offers a scalable NoSQL database.

- ⊗ Use an Elastic Load Balancer with an Auto Scaling group of Amazon EC2 instances to receive and process data, storing processed data in an Amazon S3 bucket.

While S3 is managed, this option requires managing EC2 instances, increasing operational overhead compared to serverless options.

- ⊗ Use Amazon API Gateway with AWS Lambda functions to receive data, process it, and store it in a Microsoft SQL Server Express database on an Amazon EC2 instance.

This option introduces EC2 instance management and SQL Server administration, increasing operational overhead and costs.

- ⊗ Use an Elastic Load Balancer with an Auto Scaling group of Amazon EC2 instances to receive and process data, storing processed data in an Amazon Elastic File System (Amazon EFS) shared file system.

This option requires managing EC2 instances and capacity planning for EFS, resulting in higher operational overhead.

Overall Explanation:

The most suitable solution is to use Amazon API Gateway with AWS Lambda functions to process incoming data and store it in DynamoDB. This architecture utilizes managed services, minimizing operational overhead and allowing for future expansion with independent components. Options involving EC2 instances and self-managed databases introduce increased operational responsibilities.

[View Discussions](#)[Ask AI](#)

Question 21

Not Attempted

Storage



A solutions architect is designing the storage architecture for a new web application used for storing and viewing engineering drawings. All application components will be deployed on the AWS infrastructure. The application design must support caching to minimize the amount of time that users wait for the engineering drawings to load. The application must be able to store petabytes of data. Which combination of storage and caching should the solutions architect use?

✓ Amazon S3 with Amazon CloudFront

Amazon S3 provides highly durable and scalable object storage capable of handling petabytes of data cost-effectively. Amazon CloudFront can be used to cache S3 content at the edge, minimizing latency for users and speeding up access to the engineering drawings. The global CloudFront edge network is ideal for caching large amounts of static media like drawings.

✗ Amazon S3 Glacier with Amazon ElastiCache

Amazon S3 Glacier is cheaper archival storage but has higher latency unsuited for frequent access. ElastiCache is not suitable for storing the actual drawings.

✗ Amazon Elastic Block Store (Amazon EBS) volumes with Amazon CloudFront

Amazon EBS provides block storage but lacks the scale and durability of S3 for large media files and is not cost-effective for petabytes of data. CloudFront provides edge caching capabilities.

✗ AWS Storage Gateway with Amazon ElastiCache

AWS Storage Gateway is designed for hybrid cloud environments, connecting on-premises storage to AWS. Since all components are on AWS, it is unnecessary. ElastiCache is not suitable for storing the actual drawings.

Overall Explanation:

Amazon S3 is suitable for storing petabytes of data and provides high durability and scalability. Amazon CloudFront is a content delivery network (CDN) that can cache the engineering drawings closer to the users, minimizing latency and improving the loading time. Therefore, the combination of S3 and CloudFront is the best choice.

 [View Discussions](#)

 [Ask AI](#)

Question 22

Not Attempted

[Monitoring and Observability](#)



An Amazon EventBridge rule targets a third-party API. The third-party API has not received any incoming traffic. A solutions architect needs to determine whether the rule conditions are being met and if the rule's target is being invoked. Which solution will meet these requirements?

✓ Check for metrics in Amazon CloudWatch in the namespace for AWS/Events.

EventBridge sends metrics to Amazon CloudWatch every minute, including the number of matched events and the number of times a target is invoked. The AWS/Events namespace contains these relevant metrics.

- ⊗ Review events in the Amazon Simple Queue Service (Amazon SQS) dead-letter queue.
SQS dead-letter queues store messages that fail to be delivered. While helpful for failure scenarios, they don't provide insight into whether the rule is being triggered in the first place.
- ⊗ Check for the events in Amazon CloudWatch Logs.
EventBridge primarily sends metrics to CloudWatch, not detailed logs of each event. CloudWatch Logs are not the primary place to observe event data for troubleshooting.
- ⊗ Check the trails in AWS CloudTrail for the EventBridge events.
CloudTrail records API calls made to AWS services. It will log EventBridge API calls, but it won't show whether the rule's target is being invoked or the details of each matched event going to the third-party API.

Overall Explanation:

The most appropriate solution is to check CloudWatch metrics in the AWS/Events namespace. These metrics provide information about the number of events matched by the rule and the number of invocations to the target. This allows the architect to determine if the rule conditions are being met and if the target is being invoked. SQS dead-letter queue only shows failure scenarios. CloudWatch logs are not populated with event data, only metrics. CloudTrail captures EventBridge API calls, but not individual event processing or target invocations.

[View Discussions](#)[Ask AI](#)

Question 23

Not Attempted

Security



A company is running a popular social media website where users can upload images. To minimize development effort, the company wants to ensure that uploaded images do not contain inappropriate content. Which solution should a solutions architect recommend?

- ⊗ Use Amazon Comprehend to detect inappropriate content. Use human review for low-confidence predictions.
Amazon Comprehend is designed for natural language processing (NLP), not image analysis. It's not suitable for detecting inappropriate content in images.
- ✔ Use Amazon Rekognition to detect inappropriate content. Use human review for low-confidence predictions.

Amazon Rekognition is a cloud-based image and video analysis service with pre-trained models for detecting inappropriate content. Human review can be used for low-confidence predictions to improve accuracy, minimizing development effort and utilizing a managed service.

- ⊗ Use Amazon SageMaker to detect inappropriate content. Use ground truth to label low-confidence predictions.

Amazon SageMaker is a comprehensive machine learning platform. While capable of this, it requires significant development effort to train and deploy a custom model, which contradicts the requirement of minimizing development effort.

- ⊗ Use AWS Fargate to deploy a custom machine learning model to detect inappropriate content. Use ground truth to label low-confidence predictions.

Deploying a custom model using AWS Fargate involves considerable development effort, including model training, infrastructure setup, and maintenance. This contradicts the requirement of minimizing development effort.

Overall Explanation:

Amazon Rekognition is the best solution because it provides a pre-built, managed service for image analysis, including the detection of inappropriate content. Using human review for low-confidence predictions allows for improved accuracy without significant development overhead. Comprehend is for NLP, and SageMaker/Fargate require more development effort than Rekognition.

 View Discussions

 Ask AI

Question 24

Not Attempted

Compute



A company has a large workload that runs every Friday evening. The workload runs on Amazon EC2 instances that are in two Availability Zones in the us-east-1 Region. Normally, the company must run no more than two instances at all times. However, the company wants to scale up to six instances each Friday to handle a regularly repeating increased workload. Which solution will meet these requirements with the LEAST operational overhead?

- ⊗ Create a reminder in Amazon EventBridge to scale the instances.

Creating an EventBridge reminder requires manual intervention each week to scale the instances, adding operational overhead.

- ✓ Create an Auto Scaling group that has a scheduled action.

An Auto Scaling group with a scheduled action allows defining specific dates/times to scale out or in. This automates the scaling process without requiring manual intervention.

⊗ Create an Auto Scaling group that uses manual scaling.

Manual scaling requires human involvement each week to adjust the number of instances, resulting in high operational overhead.

⊗ Create an Auto Scaling group that uses automatic scaling.

Automatic scaling responds to demand and may not align perfectly to scale out precisely every Friday evening without additional configuration and monitoring. Since the time is predicted, scheduled action is more suitable.

Overall Explanation:

Auto Scaling scheduled actions allow defining specific dates/times to scale out or in. This can be used to scale to 6 instances every Friday evening automatically. Scheduled scaling removes the need for manual intervention to scale up/down for the workload. EventBridge reminders and manual scaling require human involvement each week adding overhead. Automatic scaling responds to demand and may not align perfectly to scale out every Friday without additional tuning. Scheduled Auto Scaling actions provide the automation needed to scale for the weekly workload without ongoing operational overhead.

 View Discussions

 Ask AI

Question 25

Not Attempted

Security



A company is developing a REST API and has stringent security requirements for TLS usage. All API endpoints must use TLSv1.3, and the TLS certificate must be signed by a specific, publicly trusted third-party Certificate Authority (CA). Which solution ensures compliance with these requirements?

- ✔ Use a local machine to create a certificate that is signed by the third-party CA. Import the certificate into AWS Certificate Manager (ACM). Create an HTTP API in Amazon API Gateway with a custom domain. Configure the custom domain to use the certificate.

This option is correct. You must import a certificate signed by a third-party CA into ACM. ACM does not create certificates signed by 3rd party CAs. Then, configuring a custom domain for the API Gateway and associating it with the imported certificate will satisfy the requirements.

- ⊗ Create a certificate in AWS Certificate Manager (ACM) that is signed by the third-party CA. Create an HTTP API in Amazon API Gateway with a custom domain. Configure the custom domain to use the certificate.

This option is incorrect. AWS Certificate Manager (ACM) cannot directly create certificates signed by a third-party CA.

- ⊗ Use AWS Certificate Manager (ACM) to create a certificate that is signed by the third-party CA. Import the certificate into AWS Certificate Manager (ACM). Create an AWS Lambda function with a Lambda function URL. Configure the Lambda function URL to use the certificate.

This option is incorrect. You cannot create a third-party CA-signed certificate in ACM. Also, API Gateway is preferred over Lambda function URLs for REST APIs due to its enhanced features.

- ⊗ Create a certificate in AWS Certificate Manager (ACM) that is signed by the third-party CA. Create an AWS Lambda function with a Lambda function URL. Configure the Lambda function URL to use the certificate.

This option is incorrect. ACM cannot create a third-party CA-signed certificate, and Lambda Function URLs are not the ideal solution for REST APIs. API Gateway is better.

Overall Explanation:

The correct solution is to create the certificate signed by the third-party CA on a local machine, import it into ACM, and then associate it with a custom domain in API Gateway. AWS Certificate Manager (ACM) doesn't allow you to create a certificate that is signed by third-party CA; instead, you import the externally created certificate. API Gateway supports TLSv1.3 and the usage of ACM certificates. Lambda Function URLs are not the best option for production REST APIs.

 View Discussions

 Ask AI

Question 26

Not Attempted

Databases



A company hosts an application on AWS that experiences inconsistent usage patterns. The application relies on AWS Direct Connect to connect to an on-premises MySQL-compatible database, which consistently uses a minimum of 2 GiB of memory. The company plans to migrate the on-premises database to a managed AWS service and wants to leverage auto

scaling capabilities to handle unexpected workload increases with minimal administrative overhead. Which solution best meets these requirements?

- ☐ Provision an Amazon DynamoDB database with default read and write capacity settings.

DynamoDB is not MySQL-compatible and requires more hands-on management, increasing administrative overhead.

- ☐ Provision an Amazon Aurora database with a minimum capacity of 1 Aurora capacity unit (ACU).

While Aurora provides MySQL compatibility, this provisioned option requires manual resizing of instances to scale, leading to higher administrative overhead.

- ☒ Provision an Amazon Aurora Serverless v2 database with a minimum capacity of 1 Aurora capacity unit (ACU).

Aurora Serverless v2 offers auto-scaling, automatically handling inconsistent workloads and spikes without administrative intervention. It can scale down to zero when not in use, minimizing costs, and the minimum 1 ACU capacity is sufficient to replace the on-prem 2 GiB database.

- ☐ Provision an Amazon RDS for MySQL database with 2 GiB of memory.

RDS for MySQL requires manually resizing instances to scale, resulting in increased administrative overhead.

Overall Explanation:

Aurora Serverless v2 is the best choice due to its auto-scaling capabilities and minimal administrative overhead. It automatically adjusts capacity based on demand, is MySQL-compatible, and can scale down to zero when not in use. Provisioned Aurora and RDS require manual scaling, while DynamoDB isn't MySQL-compatible.

 View Discussions

 Ask AI

Question 27

Not Attempted

Cost Optimization



A company is adopting an event-driven architecture using AWS Lambda functions written in Java 11. They need to optimize these functions for reduced startup latency, particularly during scaling events, but without strict latency requirements for the applications. Cost is a significant

concern. Which approach offers the MOST cost-effective solution for minimizing cold starts and outlier latencies as the functions scale?

⊗ Configure Lambda provisioned concurrency.

Provisioned concurrency keeps Lambda functions initialized, reducing cold starts. However, it incurs charges for the reserved capacity, making it less cost-effective if strict low latency is not required.

⊗ Increase the timeout of the Lambda functions.

Increasing the timeout will not improve startup performance. It will only allow the function more time to execute if it is running close to the default timeout.

⊗ Increase the memory of the Lambda functions.

Increasing the memory allocation for Lambda functions can sometimes reduce cold starts by providing more resources. However, this increases cost and is not the most cost-effective method for reducing cold starts.

✓ Configure Lambda SnapStart.

Lambda SnapStart initializes functions when you publish a function version. Lambda takes a Firecracker microVM snapshot of the memory and disk state of the initialized execution environment, encrypts the snapshot, and caches it for low-latency access. When you invoke the function version for the first time, and as the invocations scale up, Lambda resumes new execution environments from the cached snapshot instead of initializing them from scratch, improving startup latency. SnapStart is optimized for applications without aggressive latency needs, reducing costs.

Overall Explanation:

Lambda SnapStart is the most cost-effective solution. It significantly reduces cold starts by using cached snapshots of initialized environments. Unlike provisioned concurrency, it doesn't require maintaining always-on capacity, making it more economical when extremely low latency is not a critical requirement. Increasing timeout and memory do not directly improve startup performance like SnapStart.

 View Discussions

 Ask AI

Question 28

Not Attempted

Cost Optimization



A financial services company launched a new application that uses an Amazon RDS for MySQL database to track stock market trends. The application is only needed for 2 hours at the end of

each week. The company wants to minimize the cost of running the database. Which solution is the MOST cost-effective for this scenario?

- ☒ Migrate the existing RDS for MySQL database to an Aurora Serverless v2 MySQL database cluster.

Aurora Serverless v2 automatically scales compute capacity based on actual usage, including scaling down to zero when not in use. It is ideal for infrequent, intermittent workloads, and is charged per second when active, making it the most cost-effective option.

- ☐ Migrate the existing RDS for MySQL database to an Aurora MySQL database cluster.

Aurora MySQL cluster requires a minimum capacity and will continue to run even when the application is not in use, incurring unnecessary costs for the rest of the week.

- ☐ Migrate the existing RDS for MySQL database to an Amazon EC2 instance that runs MySQL. Purchase an instance reservation for the EC2 instance.

Although purchasing a reserved EC2 instance lowers the hourly cost, the instance will still run for the majority of the week when the application is not in use, leading to higher costs than Aurora Serverless v2. Self-managing MySQL also adds operational overhead.

- ☐ Migrate the existing RDS for MySQL database to an Amazon Elastic Container Service (Amazon ECS) cluster that uses MySQL container images to run tasks.

Using Amazon ECS with MySQL containers involves managing the container orchestration and will incur costs for the underlying EC2 instances or Fargate tasks, even when the application is not in use. This approach is more complex and likely more expensive than Aurora Serverless v2.

Overall Explanation:

Aurora Serverless v2 is the most cost-effective solution because it scales down to zero when not in use and charges per second when active, perfectly matching the company's requirement of running the application for only 2 hours per week. Other options incur costs even when the application is idle or require more operational overhead.

 View Discussions

 Ask AI

Question 29

Not Attempted

Databases



A company deploys its applications on Amazon Elastic Kubernetes Service (Amazon EKS) behind an Application Load Balancer in an AWS Region. The application needs to store data in a PostgreSQL database engine. The company wants the data in the database to be highly

available and needs increased capacity for read workloads, ensuring minimal operational overhead. Which solution will meet these requirements with the MOST operational efficiency?

- ☐ Create an Amazon DynamoDB database table configured with global tables.

DynamoDB is a NoSQL database, not a PostgreSQL database, and requires significant application changes. It is not designed as a direct replacement or to meet the requirement of needing a PostgreSQL engine.

- ☐ Create an Amazon RDS database with Multi-AZ deployments.

While Multi-AZ deployments provide high availability, they do not inherently increase read capacity as effectively as a cluster configuration, leading to higher operational complexity for achieving the same result.

- ☒ Create an Amazon RDS database with Multi-AZ DB cluster deployment.

RDS Multi-AZ DB cluster deployments (like Aurora PostgreSQL) offer high availability, automatic failover, and increased read capacity via read replicas. They handle replication across Availability Zones, providing operational efficiency due to native RDS management.

- ☐ Create an Amazon RDS database configured with cross-Region read replicas.

Cross-Region read replicas are suitable for disaster recovery and global read capacity, but they introduce more complexity than necessary for a single-region high availability and read scaling requirement.

Overall Explanation:

RDS Multi-AZ DB cluster deployments provide high availability, automatic failover, and increased read capacity. A Multi-AZ cluster automatically handles replicating data across AZs in a single region. This maintains operational efficiency as it is natively managed by RDS without needing external replication. Other options, like DynamoDB global tables, involve complex provisioning and application changes. RDS read replicas require manual setup and management of replication. Thus, RDS Multi-AZ clustering is purpose-built by AWS for HA PostgreSQL deployments and balancing read workloads.

 View Discussions

 Ask AI

Question 30

Not Attempted

Networking



A company is building a RESTful serverless web application on AWS using Amazon API Gateway and AWS Lambda. The application's users are geographically dispersed, and the company needs to minimize API request latency for these users. Which API Gateway endpoint type should a solutions architect implement to achieve this requirement?

☐ Private endpoint

Private endpoints are designed for internal access within a VPC and do not reduce latency for geographically distributed users accessing the application over the internet.

☐ Regional endpoint

Regional endpoints are best suited for users within the same AWS region as the API Gateway. They do not provide latency benefits for globally distributed users without additional services like CloudFront.

☐ Interface VPC endpoint

Interface VPC endpoints are used for private connectivity to AWS services within a VPC and do not help reduce latency for geographically distributed internet-based users.

☒ Edge-optimized endpoint

Edge-optimized endpoints utilize Amazon CloudFront to route requests through the nearest Point of Presence (POP) to the user, thereby reducing latency for geographically distributed users.

Overall Explanation:

Edge-optimized API Gateway endpoints are the best choice for reducing latency for geographically distributed users. These endpoints use Amazon CloudFront to route requests through the closest edge location, minimizing the distance and therefore the latency. Regional endpoints are suitable for users in the same AWS Region, while Private endpoints are for internal VPC access. Interface VPC endpoints allow private connectivity to AWS services within a VPC.

 [View Discussions](#)

 [Ask AI](#)

Question 31

Not Attempted

Security



A company hosts its website using an Amazon CloudFront distribution to deliver content. They need to ensure all client connections to the website use TLS and want to automate certificate

creation and renewal with minimal operational overhead. Which solution provides the MOST operationally efficient approach?

☐ Use a CloudFront security policy to create a certificate.

CloudFront security policies do not create TLS certificates. They are used to enforce TLS versions and ciphers.

☐ Use a CloudFront origin access control (OAC) to create a certificate.

CloudFront origin access control (OAC) is used to restrict access to the origin server, not to create TLS certificates.

☒ Use AWS Certificate Manager (ACM) to create a certificate. Use DNS validation for the domain.

AWS Certificate Manager (ACM) provides free TLS/SSL certificates and automates renewals. DNS validation is preferred for automation as it does not require manual intervention.

☐ Use AWS Certificate Manager (ACM) to create a certificate. Use email validation for the domain.

While ACM can be used with email validation, it requires manual approval for each renewal, making it less operationally efficient than DNS validation.

Overall Explanation:

The most operationally efficient solution is to use AWS Certificate Manager (ACM) with DNS validation. ACM provides free TLS/SSL certificates and automates renewals. DNS validation automates the domain ownership verification, eliminating manual steps and reducing operational overhead. CloudFront security policies and OACs do not create certificates. Email validation requires manual steps, which is not ideal for automation.

 View Discussions

 Ask AI

Question 32

Not Attempted

Databases



A company deployed a serverless application that utilizes Amazon DynamoDB as its database. Due to significant user growth, the company seeks to improve database response times from milliseconds to microseconds and cache database requests. Which solution meets these requirements with the LEAST operational overhead?

✔ Use DynamoDB Accelerator (DAX).

DAX is a fully managed, highly available, in-memory cache specifically designed for DynamoDB. It delivers significant performance improvements (milliseconds to microseconds) with minimal operational overhead. It directly addresses the need to cache requests to the database.

✘ Migrate the database to Amazon Redshift.

Amazon Redshift is a data warehouse service optimized for analytics and large datasets. It is not suitable for low-latency, transactional operations, nor does it offer the least operational overhead for the given requirements.

✘ Migrate the database to Amazon RDS.

Migrating to Amazon RDS might not improve the performance to microseconds. Also, it would introduce operational overhead associated with managing a relational database.

✘ Use Amazon ElastiCache for Redis.

Amazon ElastiCache for Redis can cache requests and potentially improve response times, but implementing and managing it introduces significantly more operational overhead compared to DynamoDB Accelerator (DAX). Furthermore, it improves caching but might not directly 'improve database response' itself to microseconds.

Overall Explanation:

DynamoDB Accelerator (DAX) provides the optimal solution. DAX is purpose-built as an in-memory cache for DynamoDB and requires minimal operational overhead to implement and manage. It directly addresses the need to improve response times to microseconds and cache database requests. Options B and C are not optimized for low latency and would require significant migrations, and Option D introduces more operational overhead without guaranteeing the required performance improvements.

 View Discussions

 Ask AI

Question 33

Not Attempted

Databases



A company runs an application that uses Amazon RDS for PostgreSQL. The application receives traffic only on weekdays during business hours. The company wants to optimize costs and reduce operational overhead based on this usage pattern. Which solution will meet these requirements?

✔ Use the Instance Scheduler on AWS to configure start and stop schedules.

The Instance Scheduler on AWS automates the starting and stopping of Amazon EC2 and Amazon RDS instances, reducing operational costs by stopping resources when not in use and starting them when needed. It minimizes operational overhead and aligns instance runtime with business hour demands.

✘ Turn off automatic backups. Create weekly manual snapshots of the database.

Turning off automatic backups and creating manual snapshots increases operational overhead and introduces risk of data loss. It is not a recommended approach.

✘ Create a custom AWS Lambda function to start and stop the database based on minimum CPU utilization.

While a Lambda function could potentially work, basing it solely on CPU utilization might not be reliable and increases complexity compared to using the Instance Scheduler.

✘ Purchase All Upfront reserved DB instances.

Purchasing All Upfront Reserved DB instances might save on costs, but it doesn't stop the database when it's not in use and offers less cost optimization than a solution that stops the instance when it's not required.

Overall Explanation:

Using the Instance Scheduler on AWS is the most efficient solution. It allows for automated starting and stopping of the RDS instance based on a schedule, aligning with the application's usage during business hours, minimizing operational overhead and optimizing costs. The other options are either less efficient, increase operational overhead, or don't fully address the requirement of stopping the database when it's not in use.

 View Discussions

 Ask AI

Question 34

Not Attempted

Compute



A company aims to run its critical applications in containers to meet scalability and availability requirements. The company wants to focus on the applications' maintenance and avoid provisioning or managing the underlying infrastructure for the containerized workloads. Which solution would best meet these requirements?

✘ Use Amazon EC2 instances and install Docker on them.

This option requires manual provisioning and management of EC2 instances, and installing and configuring Docker, adding overhead and infrastructure responsibilities.

⊗ Use Amazon Elastic Container Service (Amazon ECS) on Amazon EC2 worker nodes.

While ECS helps manage containers, it still necessitates provisioning and managing EC2 instances as worker nodes, introducing complexity compared to a serverless approach.

✔ Use Amazon Elastic Container Service (Amazon ECS) on AWS Fargate.

ECS on Fargate lets you run containers without managing the underlying infrastructure. Fargate abstracts away EC2, offering serverless compute for containers and aligning with the company's preference to avoid infrastructure management.

⊗ Use Amazon EC2 instances from an Amazon Elastic Container Service (Amazon ECS)-optimized Amazon Machine Image (AMI).

Even with an ECS-optimized AMI, this still involves provisioning and managing EC2, lacking the serverless abstraction and ease of management provided by Fargate.

Overall Explanation:

AWS Fargate is a serverless compute engine for Amazon ECS, allowing container execution without managing servers. It handles the infrastructure management, letting the company focus on application maintenance. Therefore, using ECS on Fargate meets the requirement of avoiding infrastructure provisioning and management.

 View Discussions

 Ask AI

Question 35

Not Attempted

Storage



A company runs a latency-sensitive application on-premises using locally attached storage. They are migrating the application to the AWS Cloud using a lift and shift approach and want to minimize architectural changes. Which solution meets these requirements MOST cost-effectively?

⊗ Configure an Auto Scaling group with an Amazon EC2 instance. Use an Amazon FSx for Lustre file system to run the application.

FSx for Lustre is a high-performance file system, but it is more expensive and might require application changes, which contradicts the lift and shift approach.

- ⊗ Host the application on an Amazon EC2 instance. Use an Amazon Elastic Block Store (Amazon EBS) GP2 volume to run the application.

While EBS GP2 provides good performance, GP3 offers similar or better performance at a lower cost, making it less cost-effective than GP3.

- ⊗ Configure an Auto Scaling group with an Amazon EC2 instance. Use an Amazon FSx for OpenZFS file system to run the application.

FSx for OpenZFS, like FSx for Lustre, is a higher-performance file system that is more expensive and may require application changes, not ideal for a cost-effective lift and shift.

- ✓ Host the application on an Amazon EC2 instance. Use an Amazon Elastic Block Store (Amazon EBS) GP3 volume to run the application.

EBS GP3 provides a balance of good performance and cost-effectiveness for EC2 instances and is designed to replace GP2 volumes, offering lower costs for similar performance. It supports a lift and shift approach by directly attaching block storage.

Overall Explanation:

EBS GP3 is the most cost-effective option for a lift and shift migration of a latency-sensitive application. It offers similar performance to GP2 at a lower cost and doesn't require architectural changes. Auto Scaling groups are unnecessary since the question focuses on storage and maintaining the existing application architecture. FSx for Lustre and OpenZFS are more expensive and potentially require application modifications.

 View Discussions

 Ask AI

Question 36

Not Attempted

Compute



A company runs a stateful production application on Amazon EC2 instances. The application requires at least two EC2 instances to always be running. A solutions architect needs to design a highly available and fault-tolerant architecture for the application. The solutions architect creates an Auto Scaling group of EC2 instances. Which set of additional steps should the solutions architect take to meet these requirements?

- ⊗ Set the Auto Scaling group's minimum capacity to two. Deploy one On-Demand Instance in one Availability Zone and one On-Demand Instance in a second Availability Zone.

This option only deploys a total of two instances. If one AZ fails, only one instance will be running, which doesn't satisfy the requirement of at least two running instances.

- ✔ Set the Auto Scaling group's minimum capacity to four. Deploy two On-Demand Instances in one Availability Zone and two On-Demand Instances in a second Availability Zone.

This option sets the minimum capacity to four, distributing two instances in each of two Availability Zones. This ensures that even if one Availability Zone fails, at least two instances will remain running in the other Availability Zone, satisfying the high availability and fault tolerance requirements. On-Demand instances provide consistent performance and availability.

- ✘ Set the Auto Scaling group's minimum capacity to two. Deploy four Spot Instances in one Availability Zone.

This option only uses one Availability Zone, negating fault tolerance across AZs. Also, Spot Instances are not suitable for stateful, fault-intolerant applications.

- ✘ Set the Auto Scaling group's minimum capacity to four. Deploy two On-Demand Instances in one Availability Zone and two Spot Instances in a second Availability Zone.

While this option sets the minimum to four and utilizes two Availability Zones, using Spot Instances for a stateful application is risky due to potential interruptions. On-Demand instances offer more reliability.

Overall Explanation:

The correct answer is to set the Auto Scaling group's minimum capacity to four and deploy two On-Demand Instances in each of two Availability Zones. This configuration provides high availability and fault tolerance. The minimum capacity ensures that at least two instances are always running, and distributing the instances across two Availability Zones protects against AZ failures. On-Demand instances are preferred for stateful production applications to ensure consistent uptime and availability.

 View Discussions

 Ask AI

Question 37

Not Attempted

Networking



An e-commerce company hosts its website both on-premises near the us-west-1 Region and in the AWS Cloud in the eu-central-1 Region. They use Amazon Route 53 for DNS. The company wants to minimize website load times for users globally. Which Route 53 routing policy will best meet these requirements?

- ☒ Set up a geolocation routing policy. Route traffic originating near us-west-1 to the on-premises data center and traffic originating near eu-central-1 to the eu-central-1 Region.

Geolocation routing directs users to the closest endpoint based on their geographic location, minimizing latency. Routing us-west-1 traffic to the nearby on-premises data center and eu-central-1 traffic to the eu-central-1 Region achieves this.

- ☐ Set up a simple routing policy that routes all traffic originating near eu-central-1 to eu-central-1 and all traffic originating near the on-premises data center to the on-premises data center.

While simple, this lacks the specificity of geolocation and may not effectively route all users to the closest endpoint, potentially increasing latency for some.

- ☐ Set up a latency routing policy and associate the policy with the us-west-1 Region.

Associating the latency policy only with us-west-1 would not effectively route traffic to the eu-central-1 region based on latency. The policy needs to assess latency to both regions.

- ☐ Set up a weighted routing policy. Split the traffic evenly between the eu-central-1 Region and the on-premises data center.

Weighted routing doesn't consider user location or latency, potentially directing users far from a specific endpoint to that endpoint. This could increase load times.

Overall Explanation:

Geolocation routing provides the most effective solution for minimizing load times by directing users to the closest endpoint based on their geographic location. By routing traffic from the us-west-1 region to the on-premises data center and traffic from the eu-central-1 region to AWS eu-central-1, latency is minimized.

 View Discussions

 Ask AI

Question 38

Not Attempted

Cost Optimization



A company has 5 PB of archived data on physical tapes. The company needs to preserve the data on the tapes for another 10 years for compliance purposes. The company wants to migrate to AWS in the next 6 months. The data center that stores the tapes has a 1 Gbps uplink internet connectivity. Which solution will meet these requirements MOST cost-effectively?

- ⊗ Read the data from the tapes on premises. Stage the data in a local NFS storage. Use AWS DataSync to migrate the data to Amazon S3 Glacier Flexible Retrieval.

Transferring 5PB of data over a 1Gbps connection will take significantly longer than 6 months, making this option infeasible.

- ⊗ Use an on-premises backup application to read the data from the tapes and to write directly to Amazon S3 Glacier Deep Archive.

Similar to option A, transferring 5PB of data over a 1Gbps connection will take too long.

- ✔ Order multiple AWS Snowball devices that have Tape Gateway. Copy the physical tapes to virtual tapes in Snowball. Ship the Snowball devices to AWS. Create a lifecycle policy to move the tapes to Amazon S3 Glacier Deep Archive.

Given the amount of data and the limited bandwidth, using Snowball devices for physical data transfer is the most efficient and cost-effective way to migrate the data within the given timeframe.

- ⊗ Configure an on-premises Tape Gateway. Create virtual tapes in the AWS Cloud. Use backup software to copy the physical tape to the virtual tape.

Transferring 5PB of data over a 1Gbps connection will take significantly longer than 6 months. Tape Gateway depends on network bandwidth.

Overall Explanation:

The most cost-effective solution is to use AWS Snowball devices. Given the data volume (5 PB) and limited internet bandwidth (1 Gbps), transferring the data over the internet is not feasible within the 6-month timeframe. Snowball allows for physical data transfer. The Tape Gateway integration with Snowball allows copying the physical tapes to virtual tapes, and lifecycle policies can then be used to manage the data in Glacier Deep Archive.

 View Discussions

 Ask AI

Question 39

Not Attempted

Networking



A company is deploying an application that processes large quantities of data in parallel using Amazon EC2 instances. The network architecture must be configurable to prevent groups of nodes from sharing the same underlying hardware. Which networking solution meets these requirements?

- ☒ Run the EC2 instances in a spread placement group.
A spread placement group ensures that instances are placed on distinct hardware, preventing groups of nodes from sharing the same underlying hardware.
- ☐ Group the EC2 instances in separate accounts.
Grouping instances in separate accounts does not guarantee that they will not share the same underlying hardware.
- ☐ Configure the EC2 instances with dedicated tenancy.
Dedicated tenancy ensures that the hardware is dedicated to a single customer, but it does not guarantee that instances within the same account won't share hardware.
- ☐ Configure the EC2 instances with shared tenancy.
Shared tenancy allows instances to share hardware with other AWS customers, which does not meet the requirement of preventing groups of nodes from sharing the same underlying hardware.

Overall Explanation:

A spread placement group strictly places instances across distinct underlying hardware, minimizing correlated failures and ensuring that groups of nodes do not share the same hardware. Dedicated tenancy isolates hardware at the customer level, but instances from the same account could still share hardware. Separate accounts do not prevent shared hardware, and shared tenancy explicitly allows sharing of hardware. Therefore, the correct answer is running the EC2 instances in a spread placement group.

[View Discussions](#)[Ask AI](#)

Question 40

Not Attempted

[Compute](#)

A solutions architect is designing a disaster recovery (DR) strategy to provide Amazon EC2 capacity in a failover AWS Region. Business requirements state that the DR strategy must guarantee the availability of EC2 capacity in the failover Region. Which solution will meet these requirements?

- ☐ Purchase On-Demand Instances in the failover Region.
On-Demand Instances provide flexible capacity but do not reserve capacity upfront and thus don't guarantee availability.

- ⊗ Purchase an EC2 Savings Plan in the failover Region.

Savings Plans provide discounts but do not reserve capacity.

- ⊗ Purchase regional Reserved Instances in the failover Region.

Regional Reserved Instances provide discounts but do not reserve capacity.

- ✔ Purchase a Capacity Reservation in the failover Region.

Capacity Reservations allow you to reserve a specific amount of EC2 instance capacity in a given region. This ensures that the capacity is available when needed, meeting the DR requirement.

Overall Explanation:

Capacity Reservations allocate EC2 capacity in a specific AWS Region for you to launch instances. The capacity is reserved and available to be utilized when needed, meeting the requirement to provide EC2 capacity in the failover region. Other options do not reserve capacity. On-Demand provides flexible capacity but does not reserve capacity upfront. Savings Plans and Reserved Instances provide discounts but do not reserve capacity. Capacity Reservations allow defining instance attributes like instance type, platform, Availability Zone so the reserved capacity matches the production environment.

 View Discussions

 Ask AI

Question 41

Not Attempted

Automation



A company has five organizational units (OUs) as part of its organization in AWS Organizations. Each OU correlates to the five businesses that the company owns. The company's research and development (R&D) business is separating from the company and will need its own organization. A solutions architect creates a separate new management account for this purpose. What should the solutions architect do next in the new management account?

- ⊗ Have the R&D AWS account be part of both organizations during the transition.

An account cannot be part of two AWS Organizations simultaneously. This option is not possible.

- ✓ Invite the R&D AWS account to be part of the new organization after the R&D AWS account has left the prior organization.

This is the correct approach. First, the R&D account must leave the old organization. Then, the new organization can invite it.

- ✗ Create a new R&D AWS account in the new organization. Migrate resources from the prior R&D AWS account to the new R&D AWS account.

While creating a new account and migrating is an option, the question asks what the architect should do *next* given a new management account for the R&D business was already created, implying that the existing R&D account should be moved.

- ✗ Have the R&D AWS account join the new organization. Make the new management account a member of the prior organization.

The R&D department is separating, so nesting the new organization within the old is contradictory to the objective.

Overall Explanation:

The correct approach is to first remove the R&D AWS account from the original organization. After the account has successfully left the prior organization, the new management account for the R&D business can invite the R&D AWS account to join. This ensures a clean transition. According to AWS documentation:

<https://repost.aws/knowledge-center/organizations-move-accounts>

 View Discussions

 Ask AI

Question 42

Not Attempted

Security



A company is designing a solution to capture customer activity from different web applications for analytics and predictions. Customer activity is unpredictable and can surge rapidly. The solution must integrate with these web applications and include an authorization step for security. Which solution best meets these requirements?

- ✗ Configure a Gateway Load Balancer (GWLB) in front of an Amazon Elastic Container Service (Amazon ECS) container instance that stores the information in an Amazon Elastic File System (Amazon EFS) file system. Authorization is resolved at the GWLB.

Using ECS to simply store data is an inefficient and complex solution. GWLB is not suitable for API authorization.

- ⊗ Configure an Amazon API Gateway endpoint in front of an Amazon Kinesis data stream that stores the information in an Amazon S3 bucket. Use an AWS Lambda function to resolve authorization.

While this includes API Gateway and Lambda, it misses the most appropriate Kinesis service and may not automatically save the information without additional services.

- ✓ Configure an Amazon API Gateway endpoint in front of an Amazon Kinesis Data Firehose that stores the information the company receives in an Amazon S3 bucket. Use an API Gateway Lambda authorizer to resolve authorization.

This solution uses API Gateway for integration and authorization via a Lambda authorizer. Kinesis Data Firehose handles unpredictable traffic and delivers data to S3 for storage, meeting all requirements.

- ⊗ Configure a Gateway Load Balancer (GWLB) in front of an Amazon Elastic Container Service (Amazon ECS) container instance that stores the information on an Amazon Elastic File System (Amazon EFS) file system. Use an AWS Lambda function to resolve authorization.

Similar to option A, this solution is overly complex, doesn't directly address the integration requirement, and uses GWLB inappropriately.

Overall Explanation:

The best solution is to use Amazon API Gateway with a Lambda authorizer for secure API access and integration with web applications. Amazon Kinesis Data Firehose is ideal for handling unpredictable streaming data and can deliver directly to Amazon S3 for storage. Options A and D are inefficient because they use ECS and GWLB unnecessarily for storage and authorization. Option B utilizes Kinesis Data Streams, which requires more configuration to store data.

 View Discussions

 Ask AI

Question 43

Not Attempted

Databases



An ecommerce company needs a disaster recovery (DR) solution for its Amazon RDS DB instances that run Microsoft SQL Server Enterprise Edition. The company's current recovery point objective (RPO) and recovery time objective (RTO) are 24 hours. Which solution will meet these requirements MOST cost-effectively?

- ⊗ Create a cross-Region read replica and promote the read replica to the primary instance.

Creating a cross-Region read replica provides quick failover, but incurs higher ongoing costs due to continuous replication.

- ⊗ Use AWS Database Migration Service (AWS DMS) to create RDS cross-Region replication.

AWS DMS can replicate data, but is generally used for migrations and ongoing replication is a less cost-effective DR solution compared to snapshots.

- ⊗ Use cross-Region replication every 24 hours to copy native backups to an Amazon S3 bucket.

Copying native backups to S3 could work, but using RDS automatic snapshots is more streamlined and likely cheaper than managing native backups.

- ✓ Copy automatic snapshots to another Region every 24 hours.

This is the most cost-effective solution. RDS automatic snapshots are incremental, and copying them every 24 hours meets the RPO/RTO. This option utilizes existing RDS functionality and avoids the costs of constant replication.

Overall Explanation:

Copying automatic snapshots to another Region every 24 hours is the most cost-effective solution. It leverages the existing RDS snapshot functionality, minimizes data transfer costs due to incremental snapshots, and avoids the overhead of constant replication required by read replicas or DMS. It efficiently meets the 24-hour RPO and RTO requirements.

[View Discussions](#)[Ask AI](#)

Question 44

Not Attempted

[Databases](#)

A company runs a web application on Amazon EC2 instances in an Auto Scaling group behind an Application Load Balancer that has sticky sessions enabled. The web server currently hosts the user session state. The company wants to ensure high availability and avoid user session state loss in the event of a web server outage. Which solution will meet these requirements?

- ⊗ Use an Amazon ElastiCache for Memcached instance to store the session data. Update the application to use ElastiCache for Memcached to store the session state.

Memcached doesn't offer the same level of high availability and persistence as Redis, making it less suitable for session state storage where data loss must be avoided.

- ✔ Use Amazon ElastiCache for Redis to store the session state. Update the application to use ElastiCache for Redis to store the session state.

ElastiCache for Redis provides in-memory caching with persistence options and supports replication/multi-AZ for high availability, making it suitable for storing session data and preventing loss during server outages.

- ✘ Use an AWS Storage Gateway cached volume to store session data. Update the application to use AWS Storage Gateway cached volume to store the session state.

AWS Storage Gateway is primarily for hybrid cloud storage and not optimized for low-latency session data access. The latency involved would negatively impact application performance.

- ✘ Use Amazon RDS to store the session state. Update the application to use Amazon RDS to store the session state.

While RDS provides persistence, the latency of database operations is higher compared to in-memory caching solutions like ElastiCache, making it less performant for session state management.

Overall Explanation:

The optimal solution is to use Amazon ElastiCache for Redis. Redis offers in-memory performance, persistence options, and high availability through replication or clustering. This ensures that session data is quickly accessible and protected from loss, even if a web server fails. The application needs to be updated to store session data in ElastiCache rather than locally on the web servers. Memcached lacks the HA capability. Storage Gateway is not suitable for low latency requirements. RDS is persistent but has higher latency than ElastiCache.

 View Discussions

 Ask AI

Question 45

Not Attempted

Monitoring and Observability



A company hosts more than 300 global websites and applications and needs to analyze more than 30 TB of clickstream data each day. To minimize operational overhead and ensure real-time processing, which solution should a solutions architect implement to transmit and process the clickstream data?

- ⊗ Design an AWS Data Pipeline to archive the data to an Amazon S3 bucket and run an Amazon EMR cluster with the data to generate analytics.
Data Pipeline is better suited for batch processing, not real-time streaming data ingestion. EMR for analytics is valid, but the ingestion method is not ideal for this scenario.
- ⊗ Create an Auto Scaling group of Amazon EC2 instances to process the data and send it to an Amazon S3 data lake for Amazon Redshift to use for analysis.
Managing EC2 instances at this scale introduces operational overhead. This solution is also not optimized for real-time data ingestion and processing.
- ⊗ Cache the data to Amazon CloudFront. Store the data in an Amazon S3 bucket. When an object is added to the S3 bucket, run an AWS Lambda function to process the data for analysis.
CloudFront is a CDN for caching outgoing content, not for ingesting clickstream data. While Lambda can process data, it may not be the most efficient or cost-effective solution for processing 30 TB daily.
- ✔ Collect the data from Amazon Kinesis Data Streams. Use Amazon Kinesis Data Firehose to transmit the data to an Amazon S3 data lake. Load the data in Amazon Redshift for analysis.
Kinesis Data Streams captures high volumes of real-time data. Kinesis Data Firehose efficiently transmits data to S3. Redshift provides scalable analytics.

Overall Explanation:

The most appropriate solution is to use Amazon Kinesis Data Streams to ingest the clickstream data, Amazon Kinesis Data Firehose to deliver the data to an Amazon S3 data lake, and Amazon Redshift to analyze the data. This approach offers scalability, real-time processing, and minimizes operational overhead. Kinesis is designed for high-volume streaming data, Firehose simplifies data loading to S3, and Redshift provides efficient data warehousing and analytics. Data Pipeline is not recommended for new implementations and is more suited for batch processing.

[View Discussions](#)[Ask AI](#)

Question 46

Not Attempted

[Databases](#)

A company migrated a MySQL database from its on-premises data center to an Amazon RDS for MySQL DB instance. The company sized the RDS DB instance to meet the average daily workload. Once a month, the database performance degrades significantly when the company runs queries for a large report. The company needs a solution that allows them to run these reports without impacting the performance of their daily workloads. Which solution will meet these requirements in the most cost-effective manner?

- ☒ Create a read replica of the database. Direct the report queries to the read replica.

This is the most cost-effective solution. A read replica is a copy of the database synchronized with the primary database. Directing report queries to the read replica prevents performance impact on the primary database's daily workloads.

- ☐ Create a backup of the database. Restore the backup to another DB instance. Direct the report queries to the new database.

While this would work, creating a full backup and restoring it to a new instance is a time-consuming and costly approach compared to using a read replica.

- ☐ Export the data to Amazon S3. Use Amazon Athena to query the S3 bucket.

Exporting data to S3 and using Athena would require restructuring the data and the queries, adding complexity and potentially impacting the report generation process negatively compared to working with the DB directly.

- ☐ Resize the DB instance to accommodate the additional workload.

Resizing the DB instance will handle monthly peak workload but it is not cost-effective as the resources will be underutilized for the rest of the month. It would be more cost-effective to create a Read Replica.

Overall Explanation:

Creating a read replica of the RDS MySQL database and directing the report queries to the replica is the most cost-effective and efficient solution. This avoids impacting the performance of the primary database during daily workloads. Other options like creating a full backup and restoring, exporting data to S3 for Athena, or resizing the DB instance are less optimal due to cost or complexity.

 View Discussions

 Ask AI

A company runs a containerized application using Amazon Elastic Kubernetes Service (Amazon EKS). The application consists of multiple microservices for managing customers and processing orders. The company requires a solution to efficiently route incoming requests to the appropriate microservice, optimizing for cost. Which approach is MOST cost-effective?

- ☐ Use the AWS Load Balancer Controller to provision a Network Load Balancer.

While NLBs are fast and suitable for TCP/UDP traffic, they don't offer content-based routing required to direct traffic to specific microservices based on the request content. This makes them less cost-effective for this scenario, as you might need additional mechanisms to handle the routing logic.

- ☒ Use the AWS Load Balancer Controller to provision an Application Load Balancer.

ALBs operate at Layer 7, enabling content-based routing based on HTTP headers, paths, and hostnames. This allows for efficient routing to different microservices within the EKS cluster. The AWS Load Balancer Controller simplifies the management of ALBs for EKS. This is generally the most cost effective solution.

- ☐ Use an AWS Lambda function to connect the requests to Amazon EKS.

While Lambda can integrate with EKS, using Lambda as a direct intermediary for routing requests to microservices would likely introduce unnecessary complexity and overhead. It's not the most cost-effective approach for simple routing within the cluster.

- ☐ Use Amazon API Gateway to connect the requests to Amazon EKS.

API Gateway is a powerful service for managing APIs, but it may be more feature-rich and potentially more expensive than necessary for basic routing between microservices within an EKS cluster, especially at high request volumes. An ALB is often required even with API Gateway, making this option not cost-effective.

Overall Explanation:

The most cost-effective solution is to use the AWS Load Balancer Controller to provision an Application Load Balancer (ALB). ALBs offer content-based routing, allowing you to direct traffic to specific microservices based on request information like URL paths or headers. The AWS Load Balancer Controller automates the management of ALBs within an EKS cluster, simplifying the process. While API Gateway can also route traffic, it's often more expensive and complex for this specific scenario.

 View Discussions

 Ask AI

Question 48

Not Attempted

Networking



A company uses AWS to host and sell access to copyrighted images. The company's global customer base requires fast access to these images. The company is legally obligated to deny access to users from specific countries. The company wants to minimize costs. Which solution meets these requirements?

- ☐ Use Amazon S3 to store the images. Turn on multi-factor authentication (MFA) and public bucket access. Provide customers with a link to the S3 bucket.
This option does not provide geographic restrictions and MFA is not needed in this scenario. Public bucket access is also insecure.
- ☐ Use Amazon S3 to store the images. Create an IAM user for each customer. Add the users to a group that has permission to access the S3 bucket.
This solution will be very expensive and complex to manage. It also does not offer the needed level of performance and geographic restriction.
- ☐ Use Amazon EC2 instances that are behind Application Load Balancers (ALBs) to store the images. Deploy the instances only in the countries the company services. Provide customers with links to the ALBs for their specific country's instances.
This solution will be difficult to scale and more expensive than using CloudFront and S3. Deploying instances in various countries for content distribution is not cost-effective and operationally complex.
- ☒ Use Amazon S3 to store the images. Use Amazon CloudFront to distribute the images with geographic restrictions. Provide a signed URL for each customer to access the data in CloudFront.
This is the most suitable option. S3 offers cost-effective storage, CloudFront provides fast content delivery with geographic restrictions, and signed URLs control access.

Overall Explanation:

This scenario benefits from using S3 for storage, CloudFront for fast global distribution with geographic restrictions to block users from specific countries, and signed URLs to securely control access to the content. This combination ensures performance, security, and cost-effectiveness. Cloudfront allows geographic restrictions which satisfies the requirement to deny access to users from specific countries.

[View Discussions](#)[Ask AI](#)

Question 49

Not Attempted

Databases



A solutions architect is designing a highly available Amazon ElastiCache for Redis based solution. The solutions architect needs to ensure that failures do not result in performance degradation or loss of data locally and within an AWS Region. The solution needs to provide high availability at the node level and at the Region level. Which solution will meet these requirements?

- ☒ Use Multi-AZ Redis replication groups with shards that contain multiple nodes.
Multi-AZ Redis replication groups with shards containing multiple nodes provide automatic failover between AZs, ensuring high availability at both the node and Region levels. This minimizes the impact of node failures.
- ☐ Use Redis shards that contain multiple nodes with Redis append only files (AOF) turned on.
While AOF provides data persistence, it doesn't provide automatic failover or high availability at the Region level. AOF alone isn't sufficient for node-level recovery if the underlying physical server fails.
- ☐ Use a Multi-AZ Redis cluster with more than one read replica in the replication group.
While using a Multi-AZ Redis cluster with read replicas enhances availability and read performance, the key requirement is multiple nodes in each shard to minimize the impact of node failure. Though a viable option, Option A explicitly highlights the importance of multiple nodes.
- ☐ Use Redis shards that contain multiple nodes with Auto Scaling turned on.
Auto Scaling addresses scaling based on load, but it doesn't provide the automatic failover and regional high availability required in the scenario.

Overall Explanation:

The best solution is to use Multi-AZ Redis replication groups with shards that contain multiple nodes. This configuration provides automatic failover within the Region, ensuring high availability at both the node and Region levels. Multi-AZ ensures regional availability, and multiple nodes per shard minimize the impact of individual node failures. AOF is beneficial for persistence but doesn't guarantee availability, and Auto Scaling manages capacity, not necessarily availability in case of failures. It does not ensure data is consistent

in case of an AZ outage.

<https://docs.aws.amazon.com/AmazonElastiCache/latest/dev/AutoFailover.html>

 View Discussions

 Ask AI

Question 50

Not Attempted

Compute



A company is migrating its application to AWS, utilizing Amazon EC2 On-Demand Instances. During migration testing, the team discovers that application launch and memory loading times are excessively long before the application becomes fully operational. Which solution will most effectively reduce the application launch time during subsequent testing phases?

- ☐ Launch multiple EC2 On-Demand Instances, enable auto-scaling, and ensure these instances are available for the next testing phase.

Launching multiple instances and enabling auto-scaling will increase availability but does not directly address the long launch and memory loading times. It focuses on scaling rather than optimizing individual instance startup.

- ☐ Use EC2 Spot Instances to support the application and scale it to be available during the next testing phase.

Spot Instances are cost-effective but can be terminated with little notice. They do not directly reduce the initial launch and memory loading time of the application. It focuses on cost and scalability and not on reducing launch time.

- ☒ Launch EC2 On-Demand Instances with hibernation enabled and configure EC2 Auto Scaling warm pools for the next testing phase.

Hibernation allows instances to be pre-warmed to a desired state and then stopped, retaining their in-memory state. Warm pools pre-initialize instances, keeping them ready to quickly fulfill requests. This combination significantly reduces launch times as instances are already partially or fully initialized. This is the correct answer.

- ☐ Launch EC2 On-Demand Instances with Capacity Reservations and start additional EC2 instances during the next testing phase.

Capacity Reservations ensure that capacity is available, but they do not reduce the initial launch and memory loading time of the application. It focuses on ensuring resource availability rather than reducing startup time.

Overall Explanation:

The most effective solution is to utilize EC2 On-Demand Instances with hibernation enabled and configure EC2 Auto Scaling warm pools. Hibernation allows instances to be pre-warmed and paused, preserving their in-memory state. Warm pools provide a pool of pre-initialized instances, reducing the overall launch time significantly. This approach ensures that instances are quickly available, minimizing the delay observed during initial testing.

[View Discussions](#)[Ask AI](#)

Question 51

Not Attempted

[Compute](#)

A company's applications run on Amazon EC2 instances in Auto Scaling groups. The company notices that its applications experience sudden, unpredictable traffic increases on random days of the week. The company wants to maintain application performance during these traffic increases in the most cost-effective manner. Which scaling solution should the company implement?

- ☐ Use manual scaling to change the size of the Auto Scaling group.
Manual scaling requires human intervention and is not suitable for sudden, unpredictable traffic increases. It is also not cost-effective as it requires constant monitoring.
- ☐ Use predictive scaling to change the size of the Auto Scaling group.
Predictive scaling relies on historical traffic patterns. Because the traffic increases are random and unpredictable, predictive scaling will not be effective.
- ☒ Use dynamic scaling to change the size of the Auto Scaling group.
Dynamic scaling automatically adjusts the size of the Auto Scaling group based on real-time metrics, making it suitable for handling sudden, unpredictable traffic increases. It is also the most cost-effective way to handle unpredictable traffic patterns.
- ☐ Use scheduled scaling to change the size of the Auto Scaling group.
Scheduled scaling is based on predetermined schedules. Because the traffic increases occur on random days, scheduled scaling will not effectively address the issue.

Overall Explanation:

Dynamic scaling is the most cost-effective solution because it automatically adjusts the Auto Scaling group size based on real-time traffic demands, which aligns with the

unpredictable nature of the traffic increases. Manual scaling isn't automated, predictive scaling relies on patterns which don't exist, and scheduled scaling is based on pre-defined times, which don't align with the random spikes. Dynamic scaling addresses the problem directly.

[View Discussions](#)[Ask AI](#)

Question 52

Not Attempted

Databases



An ecommerce application uses a PostgreSQL database that runs on an Amazon EC2 instance. During a monthly sales event, database usage increases and causes database connection issues for the application. The traffic is unpredictable for subsequent monthly sales events, which impacts the sales forecast. The company needs to maintain performance when there is an unpredictable increase in traffic. Which solution resolves this issue in the MOST cost-effective way?

- ☒ **Migrate the PostgreSQL database to Amazon Aurora Serverless v2.**
Aurora Serverless v2 offers auto-scaling, high availability, and can be cheaper than other options due to its on-demand scaling. It automatically starts, shuts down, and scales capacity based on application needs, making it cost-effective for unpredictable traffic. It dynamically adjusts database capacity based on application needs, avoiding the costs associated with over-provisioning.
- ☐ **Enable auto scaling for the PostgreSQL database on the EC2 instance to accommodate increased usage.**
While auto-scaling the EC2 instance is possible, directly auto-scaling the PostgreSQL database on EC2 is more complex and may not be as efficient or cost-effective as a managed service.
- ☐ **Migrate the PostgreSQL database to Amazon RDS for PostgreSQL with a larger instance type.**
Migrating to a larger RDS instance type does not address the unpredictable scaling needs. It might require over-provisioning to handle peak loads, leading to unnecessary costs.
- ☐ **Migrate the PostgreSQL database to Amazon Redshift to accommodate increased usage.**
Amazon Redshift is an OLAP data warehouse service designed for analytics, not an OLTP database suitable for an ecommerce application. It is not appropriate for handling transactional workloads.

Overall Explanation:

Aurora Serverless v2 is the most cost-effective solution because it automatically scales database capacity based on application needs. This avoids the costs of over-provisioning associated with fixed-size instances, and provides high availability during unpredictable traffic spikes. Redshift is not designed for this type of transactional workload. Scaling the database manually on EC2 is complex. RDS with a larger instance type is less dynamic and may require over-provisioning.

[View Discussions](#)[Ask AI](#)

Question 53

Not Attempted

[Compute](#)

A company hosts an internal serverless application on AWS using Amazon API Gateway and AWS Lambda. Employees are experiencing high latency when they first use the application each day. The company wants to reduce this initial latency. Which solution will meet these requirements with minimal operational overhead?

- ☐ Increase the API Gateway throttling limit.
Increasing the API Gateway throttling limit focuses on throughput, not the initial latency caused by Lambda cold starts.
- ☒ Set up a scheduled scaling to increase Lambda provisioned concurrency before employees begin to use the application each day.
Provisioned concurrency pre-initializes execution environments for Lambda functions, significantly reducing cold start latency at the start of each day when employees begin using the application. This is a cost-effective and efficient solution.
- ☐ Create an Amazon CloudWatch alarm to initiate a Lambda function as a target for the alarm at the beginning of each day.
This option involves a more complex setup and might not directly address the cold start issue as effectively as provisioned concurrency. It also requires developing and maintaining an additional Lambda function.
- ☐ Increase the Lambda function memory.
Increasing Lambda function memory can sometimes improve performance but does not directly address the cold start latency issue experienced when functions are invoked for the first time after a period of inactivity.

Overall Explanation:

Provisioned concurrency pre-initializes execution environments for your functions. These execution environments are prepared to respond immediately to incoming function requests at the start of the day. By scheduling this scaling, you can pre-warm Lambda functions before peak usage times, reducing cold start latency. This solution directly addresses the latency issue caused by cold starts.

[View Discussions](#)[Ask AI](#)

Question 54

Not Attempted

Cost Optimization



A research company uses on-premises devices to generate data for analysis. The company wants to use the AWS Cloud to analyze the data. The devices generate .csv files and support writing the data to an SMB file share. Company analysts must be able to use SQL commands to query the data. The analysts will run queries periodically throughout the day. Which combination of steps will meet these requirements MOST cost-effectively? (Choose three.)

- ☒ Deploy an AWS Storage Gateway on premises in Amazon S3 File Gateway mode.
This allows the on-premises devices to write data to Amazon S3 using the SMB protocol.
- ☐ Deploy an AWS Storage Gateway on premises in Amazon FSx File Gateway mode.
This option is not ideal because the requirement is to analyze the data using SQL commands with Athena which requires data in S3.
- ☒ Set up an AWS Glue crawler to create a table based on the data that is in Amazon S3.
This allows AWS Glue to discover the schema of the .csv files in Amazon S3 and create a table definition in the AWS Glue Data Catalog.
- ☐ Set up an Amazon EMR cluster with EMR File System (EMRFS) to query the data that is in Amazon S3. Provide access to analysts.
This is not cost effective. Athena is a better choice because it is serverless and only charged for the queries run.

- ☐ Set up an Amazon Redshift cluster to query the data that is in Amazon S3. Provide access to analysts.

This is not cost effective. Athena is a better choice because it is serverless and only charged for the queries run.

- ☒ Setup Amazon Athena to query the data that is in Amazon S3. Provide access to analysts.

This allows analysts to use SQL commands to query the data in Amazon S3 using the table definition created by AWS Glue.

Overall Explanation:

The most cost-effective solution is to use AWS Storage Gateway in Amazon S3 File Gateway mode to move the data to S3, use AWS Glue to catalog the .csv data, and then use Amazon Athena to query the data using SQL. This combination avoids the overhead and cost of running a persistent EMR or Redshift cluster.

 View Discussions

 Ask AI

Question 55

Not Attempted

Compute



A company plans to run a payment processing application using Amazon ECS clusters and Amazon RDS DB instances within its on-premises data center for compliance. They intend to leverage AWS Outposts as part of their solution. The solutions architect is collaborating with the company's operational team on this project. Which of the following activities are the *responsibility* of the company's operational team in this scenario? (Choose three.)

- ☒ Providing resilient power and network connectivity to the Outposts racks.
The company is responsible for providing reliable power and network connections to ensure the Outposts racks function correctly.
- ☐ Managing the virtualization hypervisor, storage systems, and the AWS services that run on Outposts.
AWS manages the virtualization hypervisor, storage systems and AWS services that run on the Outposts.
- ☒ Physical security and access controls of the data center environment.

The company is responsible for maintaining the physical security and access controls of the data center where the Outposts racks are located.

- ⊗ Availability of the Outposts infrastructure including the power supplies, servers, and networking equipment within the Outposts racks.

AWS is responsible for the availability of the equipment within the Outposts racks.

- ⊗ Physical maintenance of Outposts components.

AWS handles the physical maintenance of the Outposts components, not the customer's operational team. However, the company is responsible for making sure its environment meets AWS Outposts' requirements, like environmental control.

- ✓ Providing extra capacity for Amazon ECS clusters to mitigate server failures and maintenance events.

The company is responsible for providing capacity on the Outpost.

Overall Explanation:

When using AWS Outposts, the customer is responsible for providing resilient power and network connectivity to the Outposts racks, maintaining physical security and access controls of the data center, and ensuring sufficient capacity to mitigate server failures and maintenance events. AWS manages the underlying infrastructure and services. Therefore, providing resilient power and network connectivity (A), physical security and access controls (C), and providing extra capacity (F) are the responsibilities of the company's operational team.

 View Discussions

 Ask AI

Question 56

Not Attempted

Storage



A company uses NFS to store large video files in on-premises network attached storage. Each video file ranges in size from 1 MB to 500 GB. The total storage is 70 TB and is no longer growing. The company decides to migrate the video files to Amazon S3. The company must migrate the video files as soon as possible while using the least possible network bandwidth. Which solution will meet these requirements?

- ⊗ Create an S3 bucket. Create an IAM role that has permissions to write to the S3 bucket. Use the AWS CLI to copy all files locally to the S3 bucket.

Using the AWS CLI to copy the files would consume a significant amount of network bandwidth and could take a long time.

- ✔ Create an AWS Snowball Edge job. Receive a Snowball Edge device on premises. Use the Snowball Edge client to transfer data to the device. Return the device so that AWS can import the data into Amazon S3.

This option minimizes network bandwidth usage and can transfer the data relatively quickly, given the large dataset and the constraint of minimal bandwidth usage. It leverages a physical device for transport.

- ✘ Deploy an S3 File Gateway on premises. Create a public service endpoint to connect to the S3 File Gateway. Create an S3 bucket. Create a new NFS file share on the S3 File Gateway. Point the new file share to the S3 bucket. Transfer the data from the existing NFS file share to the S3 File Gateway.

S3 File Gateway would utilize the internet to transfer data, which, while potentially providing NFS access, would not minimize bandwidth usage, especially for a large dataset such as 70TB.

- ✘ Set up an AWS Direct Connect connection between the on-premises network and AWS. Deploy an S3 File Gateway on premises. Create a public virtual interface (VIF) to connect to the S3 File Gateway. Create an S3 bucket. Create a new NFS file share on the S3 File Gateway. Point the new file share to the S3 bucket. Transfer the data from the existing NFS file share to the S3 File Gateway.

While Direct Connect provides a dedicated connection, using it with S3 File Gateway still requires data transfer over the network, not fulfilling the least possible bandwidth usage requirement and adding unnecessary complexity with S3 File Gateway.

Overall Explanation:

The best solution is to use AWS Snowball Edge. It minimizes the use of network bandwidth by physically transporting the data to AWS. While it introduces a delay for shipping, it aligns best with the requirement of 'least possible network bandwidth' and can be faster overall than transferring 70TB of data over a network, especially when bandwidth is a concern. S3 File Gateway and AWS CLI would require substantial network bandwidth, and Direct Connect, while faster than a standard internet connection, still uses network bandwidth and is more complex than Snowball Edge for this specific one-time migration scenario. The fact that the data is no longer growing also supports using Snowball Edge for a one-time transfer.

Question 57

Not Attempted

Networking



A company hosts its website on AWS, using an Application Load Balancer (ALB) configured to handle HTTP and HTTPS traffic separately. To enhance security, the company wants to ensure all requests are served over HTTPS. What is the MOST efficient way for a Solutions Architect to meet this requirement?

- ☐ Update the ALB's security group to only allow HTTPS traffic on port 443.
Updating the security group will restrict HTTP traffic, but it won't redirect existing HTTP requests to HTTPS. Users accessing the site via HTTP will receive an error instead of being redirected.
- ☐ Implement a custom Lambda function to inspect each request and redirect HTTP traffic to HTTPS.
Using a Lambda function for redirection adds unnecessary complexity and latency to the request processing. It's not the most efficient solution as the ALB can handle redirection directly.
- ☒ Create a listener rule on the ALB to redirect HTTP traffic (port 80) to HTTPS traffic (port 443).
Creating a listener rule is the most efficient way. The ALB can redirect HTTP traffic to the HTTPS listener based on pre-configured rules.
- ☐ Replace the ALB with a Network Load Balancer (NLB) and configure it to terminate TLS connections.
Replacing the ALB with an NLB is unnecessary and more complex. While an NLB can terminate TLS, it doesn't provide the same level of HTTP/HTTPS awareness and redirection capabilities as an ALB.

Overall Explanation:

The most efficient solution is to create a listener rule on the Application Load Balancer (ALB) to redirect HTTP traffic to HTTPS. This allows the ALB to automatically handle the redirection without requiring any code changes or infrastructure modifications. Network ACLs operate at the subnet level and cannot distinguish between HTTP and HTTPS requests to implement a redirect. Replacing HTTP with HTTPS in the URL happens at the client side, it does not redirect at the ALB. Network Load Balancers work at the TCP level and do not understand HTTP or HTTPS protocols, so they cannot redirect in this manner.

[View Discussions](#)[Ask AI](#)

Question 58

Not Attempted

Networking



A company is migrating a TCP-based application to its VPC. Currently, the application is publicly accessible through a hardware appliance in the company's data center, using a nonstandard TCP port. This endpoint handles up to 3 million requests per second with low latency. The company needs to maintain the same performance level for the new public endpoint in AWS. Which solution should a solutions architect recommend?

- ☒ Deploy a Network Load Balancer (NLB). Configure the NLB to be publicly accessible over the TCP port that the application requires.
A Network Load Balancer (NLB) operates at the transport layer (Layer 4) of the OSI model and can handle millions of requests per second. It's suitable for TCP-based applications requiring high performance and low latency. By configuring it to be publicly accessible on the required TCP port, the company can meet the performance requirements.
- ☐ Deploy an Application Load Balancer (ALB). Configure the ALB to be publicly accessible over the TCP port that the application requires.
An Application Load Balancer (ALB) is designed for HTTP/HTTPS traffic and is not suitable for handling general TCP traffic on nonstandard ports with high throughput requirements.
- ☐ Deploy an Amazon CloudFront distribution that listens on the TCP port that the application requires. Use an Application Load Balancer as the origin.
Amazon CloudFront is a CDN designed for caching and delivering content closer to users. It's not designed for handling general TCP traffic or acting as a primary endpoint for an application requiring very high throughput on a nonstandard port. Also, it is mainly used for web content and APIs.
- ☐ Deploy an Amazon API Gateway API that is configured with the TCP port that the application requires. Configure AWS Lambda functions with provisioned concurrency to process the requests.
Amazon API Gateway is primarily designed for HTTP-based APIs and is not suitable for handling general TCP traffic directly on a nonstandard port. While Lambda functions can process requests, API Gateway is not designed for the extreme throughput (3 million requests per second) specified in the question.

Overall Explanation:

The Network Load Balancer (NLB) is the most suitable solution. NLBs operate at Layer 4 (TCP) and are designed to handle millions of requests per second with low latency. The other options (ALB, CloudFront, API Gateway) are not designed for this type of TCP-based application with such high performance requirements. Therefore, deploying an NLB configured to be publicly accessible over the required TCP port provides the same level of performance as the existing hardware appliance.

[View Discussions](#)[Ask AI](#)

Question 59

Not Attempted

Databases



A company's critical database runs on an Amazon RDS for PostgreSQL DB instance. The company needs to migrate to Amazon Aurora PostgreSQL with the lowest possible downtime and minimal data loss. Which solution meets these requirements while also minimizing operational overhead?

- ☐ Create a DB snapshot of the RDS for PostgreSQL DB instance to populate a new Aurora PostgreSQL DB cluster.
Using a DB snapshot requires manually restoring data, which increases downtime and operational overhead.
- ☒ Create an Aurora read replica of the RDS for PostgreSQL DB instance. Promote the Aurora read replica to a new Aurora PostgreSQL DB cluster.
Creating an Aurora read replica allows replication from RDS PostgreSQL to Aurora PostgreSQL with minimal downtime. Promoting the read replica to a full Aurora DB cluster incurs very little downtime. This leverages AWS's managed replication.
- ☐ Use data import from Amazon S3 to migrate the database to an Aurora PostgreSQL DB cluster.
Data import from S3 requires exporting, uploading, and importing data, which adds operational overhead.
- ☐ Use the pg_dump utility to back up the RDS for PostgreSQL database. Restore the backup to a new Aurora PostgreSQL DB cluster.
Using pg_dump and restore requires manually managing backups and restoring data, increasing downtime and operational overhead.

Overall Explanation:

The Aurora read replica option minimizes downtime during the migration and leverages AWS's managed replication. Creating a read replica involves nearly no data loss and downtime. Other options introduce data loss and more downtime.

[View Discussions](#)[Ask AI](#)

Question 60

Not Attempted

Storage



A company's infrastructure consists of hundreds of Amazon EC2 instances that use Amazon Elastic Block Store (Amazon EBS) storage. A solutions architect must ensure that every EC2 instance can be recovered after a disaster. What should the solutions architect do to meet this requirement with the LEAST amount of effort?

- ☐ Take a snapshot of the EBS storage that is attached to each EC2 instance. Create an AWS CloudFormation template to launch new EC2 instances from the EBS storage.
Manually managing templates for hundreds of EC2 instances requires too much effort.
- ☐ Take a snapshot of the EBS storage that is attached to each EC2 instance. Use AWS Elastic Beanstalk to set the environment based on the EC2 template and attach the EBS storage.
Beanstalk is designed for managing application environments, not backups.
- ☒ Use AWS Backup to set up a backup plan for the entire group of EC2 instances. Use the AWS Backup API or the AWS CLI to speed up the restore process for multiple EC2 instances.
AWS Backup automates backups and allows programmatic restores, minimizing effort.
- ☐ Create an AWS Lambda function to take a snapshot of the EBS storage that is attached to each EC2 instance and copy the Amazon Machine Images (AMIs). Create another Lambda function to perform the restores with the copied AMIs and attach the EBS storage.
Creating custom Lambda functions adds complexity and effort when AWS Backup provides a managed solution.

Overall Explanation:

AWS Backup offers automated backup scheduling and retention management for EC2 instances and EBS volumes. It allows defining backup policies for groups of resources, removing the need to manually create and manage backups for each instance. The AWS Backup API and CLI enable programmatic control of backup plans and restores, facilitating the efficient recovery of hundreds of EC2 instances after a disaster. This option provides the least amount of effort compared to manually creating snapshots or using custom Lambda functions.

[View Discussions](#)[Ask AI](#)

Question 61

Not Attempted

[Compute](#)

A company recently migrated to the AWS Cloud and wants a serverless solution for large-scale parallel on-demand processing of a semistructured dataset. The data, consisting of logs, media files, sales transactions, and IoT sensor data, is stored in Amazon S3. The solution must be able to process thousands of items in the dataset in parallel with the MOST operational efficiency. Which solution meets these requirements?

- ☐ Use the AWS Step Functions Map state in Inline mode to process the data in parallel.
The Step Functions Map state in Inline mode has concurrency limits that might not be suitable for processing thousands of items in parallel. It supports concurrency of 40 parallel branches and execution history limits of 25,000 events or approximately 6,500 state transitions in a workflow.
- ☒ Use the AWS Step Functions Map state in Distributed mode to process the data in parallel.
The Step Functions Map state in Distributed mode is optimized for large-scale parallel processing of data in Amazon S3 and can handle up to 10,000 parallel branches. It is a serverless solution that automatically scales resources based on demand, providing high operational efficiency.
- ☐ Use AWS Glue to process the data in parallel.
While AWS Glue is a serverless data integration service capable of processing data in parallel, it is primarily designed for ETL (extract, transform, load) operations and might not be as efficient as Step Functions Distributed Map for simple on-demand parallel processing of S3 objects.
- ☐ Use several AWS Lambda functions to process the data in parallel.
While Lambda functions can process data in parallel, orchestrating and managing thousands of Lambda functions can be complex and less operationally efficient than using Step Functions Distributed Map.

Overall Explanation:

AWS Step Functions with the Map state in Distributed mode is the MOST operationally efficient solution because it's serverless, scales automatically to process thousands of items in parallel directly from S3, and requires minimal management overhead. The Distributed Map has been optimized for Amazon S3.

[View Discussions](#)[Ask AI](#)

Question 62

Not Attempted

Networking



A company plans to migrate 10 PB of data to Amazon S3 within 6 weeks. Their current data center has a 500 Mbps internet uplink, which is shared with other on-premises applications. The company can dedicate 80% of the internet bandwidth for this one-time migration. Which solution is the MOST suitable to meet these requirements?

- ☐ **Configure AWS DataSync to migrate the data to Amazon S3 and automatically verify the data.**
Using DataSync over a constrained bandwidth would take an impractically long time to transfer 10 PB of data.
- ☐ **Use rsync to transfer the data directly to Amazon S3.**
Similar to DataSync, rsync over the internet uplink would take too long to transfer the data within the given timeframe.
- ☐ **Use the AWS CLI and multiple copy processes to send the data directly to Amazon S3.**
Using the AWS CLI directly would also be limited by the internet uplink's bandwidth and would not meet the time constraint.
- ☒ **Order multiple AWS Snowball devices. Copy the data to the devices. Send the devices to AWS to copy the data to Amazon S3.**
Snowball devices are designed for large-scale data transfers when network bandwidth is limited. This option avoids reliance on the internet connection and is the most feasible option.

Overall Explanation:

Given the large data volume (10 PB) and the limited bandwidth available (80% of 500 Mbps), using AWS Snowball is the most effective approach. Transferring 10PB of data over

a 400 Mbps connection would take years. Snowball allows for offline data transfer, bypassing the bandwidth constraint and meeting the 6-week deadline. The other options rely on the network connection and would take significantly longer. Therefore, ordering multiple Snowball devices, copying the data to them, and shipping them to AWS is the best option.

[View Discussions](#)[Ask AI](#)

Question 63

Not Attempted

Storage



A company wants to reduce the number of on-premises Internet Small Computer Systems Interface (iSCSI) network storage servers by migrating to the AWS Cloud. They require low-latency access to frequently used data, minimal infrastructure changes, and reduced dependency on the on-premises servers. A solutions architect needs to design a solution that meets these requirements. Which solution should the architect implement?

- ☐ **Deploy an Amazon S3 File Gateway.**
Amazon S3 File Gateway is for file storage, not block storage via iSCSI. Therefore, it is not the best solution for replacing iSCSI servers.
- ☐ **Deploy Amazon Elastic Block Store (Amazon EBS) storage with backups to Amazon S3.**
EBS does not natively provide iSCSI connectivity for on-premises servers to connect to it directly as a storage target.
- ☐ **Deploy an AWS Storage Gateway volume gateway that is configured with stored volumes.**
A volume gateway configured with stored volumes would store all data on-premises first, which does not reduce the dependency on on-premises servers.
- ☒ **Deploy an AWS Storage Gateway volume gateway that is configured with cached volumes.**
A volume gateway configured with cached volumes stores frequently accessed data locally for low latency access and asynchronously backs up all data to AWS S3. This approach allows replacing the on-premises iSCSI servers with minimal changes, providing low latency to frequently accessed data, while storing the rest of the data in S3.

Overall Explanation:

The best solution is to deploy an AWS Storage Gateway volume gateway configured with cached volumes. This allows maintaining low-latency access to frequently accessed data locally while storing the rest of the data in S3, reducing dependency on on-premises servers and allowing for a relatively easy migration from existing iSCSI infrastructure. Stored volumes keep all data on-premise and S3 File Gateway is for file based access.

[View Discussions](#)[Ask AI](#)

Question 64

Not Attempted

Storage



A solutions architect is designing an application that will allow business users to upload objects to Amazon S3. The solution needs to maximize object durability. Objects also must be readily available at any time and for any length of time. Users will access objects frequently within the first 30 days after the objects are uploaded, but users are much less likely to access objects that are older than 30 days. Which solution meets these requirements MOST cost-effectively?

- ☐ Store all the objects in S3 Standard with an S3 Lifecycle rule to transition the objects to S3 Glacier after 30 days.

While S3 Glacier is cost-effective for archival, it's not ideal for objects that need to be readily available. Retrieval times can be longer, increasing costs if older objects need to be accessed.

- ☒ Store all the objects in S3 Standard with an S3 Lifecycle rule to transition the objects to S3 Standard-Infrequent Access (S3 Standard-IA) after 30 days.

This option provides high durability with immediate availability using S3 Standard for the first 30 days, and then cost savings using S3 Standard-IA for less frequently accessed objects after 30 days. This meets the requirements effectively and economically.

- ☐ Store all the objects in S3 Standard with an S3 Lifecycle rule to transition the objects to S3 One Zone-Infrequent Access (S3 One Zone-IA) after 30 days.

S3 One Zone-IA does not maximize object durability because the data is stored in a single Availability Zone. This makes it less suitable when durability is a key requirement.

- ☐ Store all the objects in S3 Intelligent-Tiering with an S3 Lifecycle rule to transition the objects to S3 Standard-Infrequent Access (S3 Standard-IA) after 30 days.

S3 Intelligent-Tiering automatically transitions data between frequent and infrequent access tiers based on access patterns. Adding a lifecycle policy to *force* transition after 30 days is redundant and less cost effective than using standard lifecycle policies.

Overall Explanation:

The most cost-effective solution that maximizes durability and ensures objects are readily available is to store objects in S3 Standard initially and then transition them to S3 Standard-IA after 30 days using a Lifecycle rule. S3 Standard provides high durability and frequent access, while S3 Standard-IA offers cost savings for infrequent access while maintaining high durability.

[View Discussions](#)[Ask AI](#)

Question 65

Not Attempted

[Databases](#)

A company has migrated a two-tier application from its on-premises data center to the AWS Cloud. The data tier is a Multi-AZ deployment of Amazon RDS for Oracle with 12 TB of General Purpose SSD Amazon Elastic Block Store (Amazon EBS) storage. The application is designed to process and store documents in the database as binary large objects (blobs) with an average document size of 6 MB. The database size has grown significantly over time, impacting performance and increasing storage costs. The company needs to improve database performance and ensure high availability and resilience while maintaining cost-effectiveness. Which solution is the MOST suitable?

- ☐ Reduce the RDS DB instance size. Increase the storage capacity to 24 TiB. Change the storage type to Magnetic.

Reducing the instance size might negatively impact performance. Increasing storage capacity and switching to Magnetic storage is not a cost-effective way to improve performance, and Magnetic storage is generally slower than SSD.

- ☐ Increase the RDS DB instance size. Increase the storage capacity to 24 TiB. Change the storage type to Provisioned IOPS.

Increasing instance size and provisioning more IOPS can improve performance, but it doesn't address the fundamental issue of storing large blobs in the database, which is inefficient and costly. Provisioned IOPS can be expensive.

- ☒ Create an Amazon S3 bucket. Update the application to store documents in the S3 bucket. Store the object metadata in the existing database.

This solution is the most cost-effective and efficient. Storing the large documents in S3 optimizes the RDS database by reducing its size and improving its performance. S3 is designed for object

storage and provides excellent scalability, durability, and cost-effectiveness. Storing metadata in the database maintains relational integrity.

- ⊗ Create an Amazon DynamoDB table. Update the application to use DynamoDB. Use AWS Database Migration Service (AWS DMS) to migrate data from the Oracle database to DynamoDB.

DynamoDB is not designed for storing large binary objects. DynamoDB has a size limit of 400KB per item, making it unsuitable for storing 6MB documents.

Overall Explanation:

Storing BLOBs in a database is generally less efficient and more expensive than storing them in an object storage service like Amazon S3. Option C is the most cost-effective solution because it leverages S3's scalability and cost-efficiency for storing the documents, while keeping metadata in the existing RDS Oracle database for relational integrity. Options A and B don't address the BLOB storage issue, and option D is incorrect due to DynamoDB's size limitations for storing large objects.

[View Discussions](#)[Ask AI](#)[← Back to Exam](#)

Marketing

Company

Affiliate Program

Udemy Alternative

Unlimited Access

Whizlabs Alternative

Business Unlimited Access

TutorialsDojo Alternative

Support

Legal

[Contact Us](#)

[Terms of service](#)

team@itcertifications.io

[Privacy policy](#)

[Refund policy](#)

[Shipping policy](#)



© 2026, ITcertifications Technologies. All rights reserved.